
Software Requirements Specification

for

Library Management System

Version 1.0 approved

Prepared by Iyyappan R

Department of Computer Science and Engineering

15 - 10 - 2026

Table of Contents

Table of Contents	ii
Revision History	ii
1. Introduction	1
1.1 Purpose	1
1.2 Document Conventions	1
1.3 Intended Audience and Reading Suggestions.....	1
1.4 Product Scope	1
1.5 References	2
2. Overall Description	2
2.1 Product Perspective	2
2.2 Product Functions	2
2.3 User Classes and Characteristics	2
2.4 Operating Environment	7
2.5 Design and Implementation Constraints.....	8
2.6 User Documentation	8
2.7 Assumptions and Dependencies	8
3. External Interface Requirements	8
3.1 User Interfaces	8
3.2 Hardware Interfaces	9
3.3 Software Interfaces	9
3.4 Communications Interfaces	9
4. System Features	9
4.1 User Authentication and Role-Based Access Control	9
4.2 User and Profile Management	10
4.3 Book Management.....	10
4.4 Book Search and Catalog	11
4.5 Book Issue Management	11
4.6 Book Return and Renewal.....	11
4.7 Borrowing History Management	12
4.8 Faculty Book Request.....	12
4.9 Department Resource Management.....	13
4.10 Fine and Overdue Monitoring	13
4.11 Report And Analysis	14
5. Other Nonfunctional Requirements	14
5.1 Performance Requirements.....	14
5.2 Safety Requirements.....	14
5.3 Security Requirements.....	15
5.4 Software Quality Attributes.....	15
5.5 Business Rules.....	15
6. Other Requirements	15
Appendix A: Glossary.....	16
Appendix B: Analysis Models	17
Appendix C: To Be Determined List.....	41

Revision History

Name	Date	Reason For Changes	Version

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) document describes the functional and non-functional requirements for the Library Management System (LMS), Version 1.0. The LMS is a web-based software application developed to automate and streamline the daily operations of a library. This document defines the complete scope of the system, including book management, member management, book issuing and returning, book search, overdue monitoring, and report generation. It serves as the primary reference for the design, development, testing, implementation, and maintenance of the Library Management System.

1.2 Document Conventions

This document follows the IEEE Std 830-1998 standard for Software Requirements Specifications. The following conventions apply: (1) Requirements are written using the keyword SHALL for mandatory requirements and SHOULD for recommended features. (2) Each functional requirement is assigned a unique identifier in the format FR-XX. (3) Non-functional requirements are prefixed with NFR-XX. (4) Section headings follow a hierarchical decimal numbering system. (5) Bold text is used to highlight critical terms and important definitions. (6) User roles such as Administrator, Librarian, and Library Member are explicitly identified throughout the document. (7) Higher-level requirements are assumed to propagate their priority levels to their child requirements unless stated otherwise. (8) All dates, book records, and member information referenced in the system shall follow a standardized format to ensure consistency and ease of maintenance throughout the application.

1.3 Intended Audience and Reading Suggestions

This SRS document is intended for the following audiences: (1) Software Developers and Engineers – responsible for designing and implementing the Library Management System modules; they should read all sections thoroughly. (2) Project Managers – responsible for project planning, scheduling, and progress tracking; they should focus on Sections 1, 2, and 5. (3) System Testers and QA Engineers – responsible for verifying that the system satisfies all specified requirements; they should focus on Sections 3 and 4. (4) Librarians and Library Administrators – end-users who need to understand the system capabilities and workflows; they should read Sections 1, 2, and 4. (5) Database Administrators – responsible for database design, maintenance, and data management; they should focus on the data requirements in Section 4. (6) Library Members or Students – users who access the system to search books, check availability, and view borrowing records; they should focus on the relevant user functionalities described in Section 4. It is recommended that all readers begin with Section 1 and Section 2 to gain a comprehensive understanding of the product before proceeding to their relevant sections.

1.4 Product Scope

The Library Management System (LMS) is a full-featured, role-based web application designed to digitize and manage all essential library operations. The primary goal of the LMS is to eliminate manual, paper-based processes and replace them with a centralized, secure, and efficient digital platform. The system enables quick access to library resources, reduces administrative errors, improves record management, and enhances the overall efficiency of library services. Key objectives include: providing a unified platform for librarians, administrators, and library members; enabling real-time access to book availability and member

records; automating book issuing and returning processes; monitoring overdue books and maintaining borrowing history; and generating insightful reports related to book circulation, library usage, and resource utilization. The LMS is intended for use in schools, colleges, universities, and public libraries seeking to modernize their operations and improve service quality.

1.5 References

The following references were consulted during the preparation of this SRS: (1) IEEE Std 830-1998, "IEEE Recommended Practice for Software Requirements Specifications," IEEE, 1998. (2) Sommerville, I., "Software Engineering," 10th Edition, Pearson Education, 2015. (3) Pressman, R. S., "Software Engineering: A Practitioner's Approach," 8th Edition, McGraw-Hill, 2014. (4) MySQL 8.0 Reference Manual, Oracle Corporation, <https://dev.mysql.com/doc/>. (5) HTML5 Documentation, World Wide Web Consortium (W3C), <https://html.spec.whatwg.org/>. (6) CSS Documentation, World Wide Web Consortium (W3C), <https://www.w3.org/TR/CSS/>. (7) JavaScript Documentation, Mozilla Developer Network (MDN), <https://developer.mozilla.org/en-US/docs/Web/JavaScript>.

2. Overall Description

2.1 Product Perspective

The Library Management System (LMS) is a new, self-contained web application developed to replace the existing manual and paper-based library management processes. Currently, many libraries rely on physical registers, spreadsheets, and handwritten records for managing books, members, and borrowing activities. The LMS consolidates all these functions into a single integrated platform accessible through a web browser. The system interacts with a centralized database for persistent storage and provides role-specific interfaces for administrators, librarians, and library members. The LMS is designed to be a standalone application but can be extended in the future to integrate with external systems such as digital libraries, student information systems, and online book repositories. A high-level system architecture may include three major components: the client layer (web browser), application layer (business logic), and data layer (database server).

2.2 Product Functions

The Library Management System shall provide the following major functions: (1) User Authentication and Role-Based Access Control – secure login with role-specific permissions for all system users. (2) Book Management – add, update, delete, and maintain detailed information about books, including title, author, category, publication details, and availability status. (3) Member Management – register new library members and maintain their profiles. (4) Book Search – search books using title, author, category, or publication details. (5) Book Issue Management – issue books to members and automatically update records. (6) Book Return Management – process returned books and update availability status. (7) Borrowing History Management – maintain records of all issued and returned books. (8) Overdue Monitoring – track overdue books and generate alerts for delayed returns. (9) Report Generation – produce daily, weekly, and monthly reports on book circulation, library usage, and popular books. (10) Inventory Management – monitor book availability and assist in future acquisition planning.

2.3 User Classes and Characteristics

The LMS serves three distinct user classes: (1) Library Administrator – highest privilege level; manages user accounts, monitors all library activities, generates reports, and configures system settings. Uses the system

daily and requires full access to all modules. (2) Librarian – manages books, registers members, issues and returns books, maintains borrowing records, and monitors overdue books. Frequent daily user with access to operational modules. (3) Library Member – searches for books, checks book availability, and views borrowing history. Occasional user with limited access to personal information and library resources.

The use case diagrams and the corresponding use case descriptions for each user class of the Library Management System are given below.

1. Student

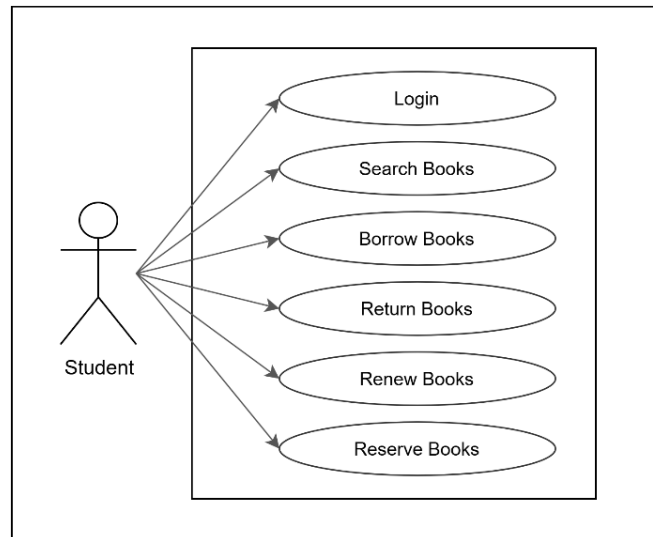


Figure 2.1: Use Case Diagram – Student

Use Case: Borrow Book

Primary Actor: Student / Member

Secondary Actor: Librarian, System

Precondition:

- Student is registered and logged in.
- Student has a valid library membership.
- Book is available.

Postcondition:

- Book is issued to the student.
- Borrowing history is updated.
- Due date is generated.

Trigger:

- Student selects the “Borrow Book” option.

Main Access Scenario:

- Student logs into the system.
- Searches for the required book.
- Selects an available book.
- Sends a borrow request.
- Librarian verifies the request.
- System issues the book.
- System updates borrowing history and due date.

Exception Scenario:

- *Book is not available.*
- *Student has exceeded the borrowing limit.*

2. Faculty

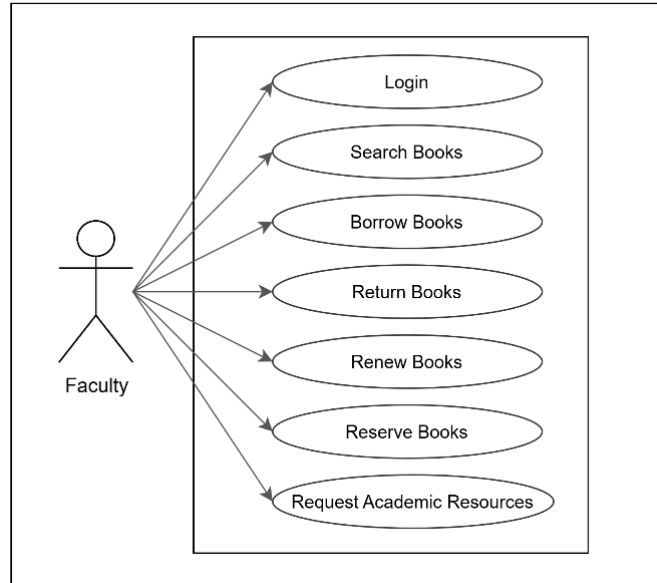


Figure 2.2: Use Case Diagram – Faculty

Use Case: *Request Academic Resources*

Primary Actor: *Faculty Member*

Secondary Actor: *Department Coordinator, Librarian, System*

Precondition:

- *Faculty member is registered and logged in.*
- *Requested book is not available in the library.*

Postcondition:

- *Book request is submitted.*
- *Department Coordinator receives the request.*

Trigger:

- *Faculty selects “Request New Academic Book.”*

Main Access Scenario:

- *Faculty logs into the system.*
- *Searches for the required academic book.*
- *Confirms the book is unavailable.*
- *Enters book details.*
- *Submits the request.*
- *System records the request.*
- *Department Coordinator reviews the request.*

Exception Scenario:

- *Requested book already exists.*
- *Required details are incomplete.*
- *Request submission fails.*

3. Librarian

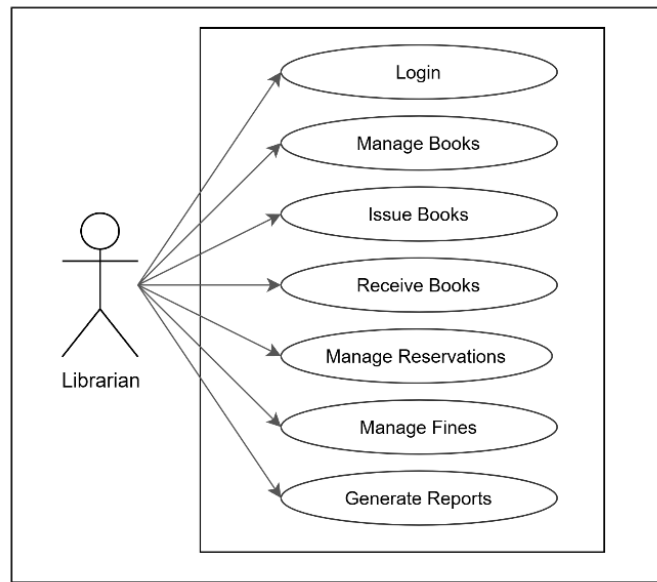


Figure 2.3: Use Case Diagram – Librarian

Use Case: *Issue Book*

Primary Actor: *Librarian*

Secondary Actor: *Student / Faculty Member, System*

Precondition:

- *Librarian is logged in.*
- *User has a valid membership.*
- *Book is available.*

Postcondition:

- *Book is issued successfully.*
- *Inventory is updated.*
- *Due date is assigned.*

Trigger:

- *Librarian selects “Issue Book.”*

Main Access Scenario:

- *Librarian logs into the system.*
- *Searches for the member.*
- *Verifies membership status.*
- *Scans or selects the book.*
- *Confirms issue transaction.*
- *System updates inventory.*
- *System records due date and borrowing details.*

Exception Scenario:

- *Membership is inactive.*
- *Book is already issued.*
- *Borrowing limit exceeded.*

4. Admin

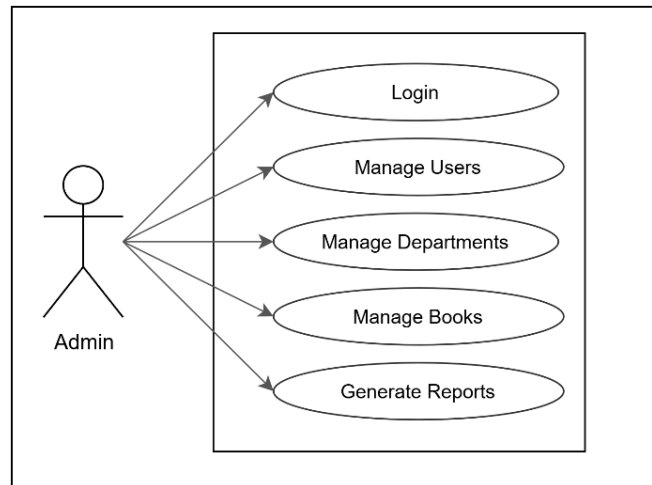


Figure 2.4: Use Case Diagram – Admin

Use Case: *Manage Users*

Primary Actor: *Administrator*

Secondary Actor: *System*

Precondition:

- *Administrator is authenticated and logged in.*

Postcondition:

- *User accounts are created, updated, disabled, or deleted.*

Trigger:

- *Administrator selects “Manage Users.”*

Main Access Scenario:

- *Administrator logs into the system.*
- *Opens the user management module.*
- *Searches for an existing user or creates a new user.*
- *Adds, updates, or removes user information.*
- *Assigns appropriate user roles.*
- *Saves the changes.*
- *System updates the user database.*

Exception Scenario:

- *Duplicate user ID.*
- *Required information is missing.*
- *Database update fails.*

5. Department Coordinator

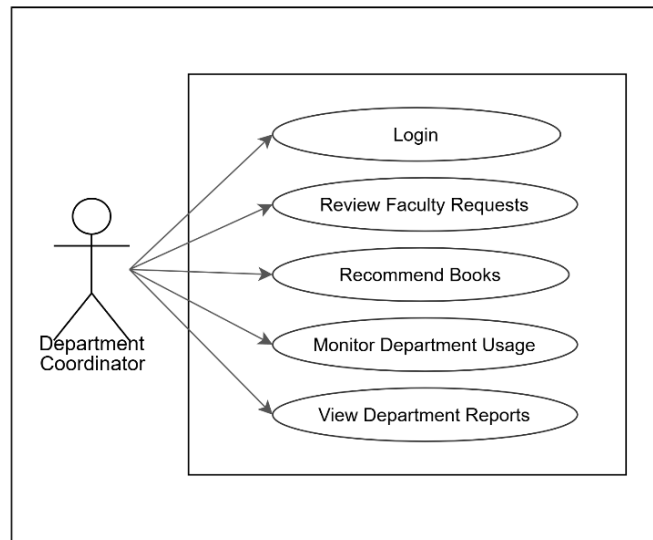


Figure 2.5: Use Case Diagram – Department Coordinator

Use Case: Review Faculty Request

Primary Actor: Department Coordinator

Secondary Actor: Faculty Member, Librarian, System

Precondition:

- Coordinator is logged in.
- Faculty has submitted a book request.

Postcondition:

- Request is approved or rejected.
- Librarian receives the decision.

Trigger:

- Coordinator opens pending faculty requests.

Main Access Scenario:

- Coordinator logs into the system.
- Views pending book requests.
- Reviews request details.
- Approves or rejects the request.
- System updates request status.
- Librarian receives notification.

Exception Scenario:

- Request already processed.
- Invalid request details.
- System fails to update status.

2.4 Operating Environment

The Library Management System shall operate in the following environment: (1) Client Side: Any modern web browser (Google Chrome v100+, Mozilla Firefox v100+, Microsoft Edge v100+, Safari v15+) running on Windows 10/11, macOS 12+, or Ubuntu 20.04+. A minimum screen resolution of 1280×720 is

recommended. (2) Server Side: A Linux-based server (Ubuntu 22.04 LTS recommended) with a minimum configuration of 4-core CPU, 8 GB RAM, and 100 GB SSD storage. (3) Database: MySQL 8.0+ for storing book records, member information, and transaction details. (4) Network: Stable internet or LAN connection with a minimum bandwidth of 10 Mbps. (5) Security: HTTPS with TLS 1.2+ shall be used to secure all client-server communications.

2.5 Design and Implementation Constraints

The following constraints shall apply to the development of the LMS: (1) The system shall be developed using standard web technologies and a relational database management system. (2) User passwords shall be securely encrypted before storage. (3) Role-Based Access Control (RBAC) shall be enforced throughout the application. (4) The user interface shall be responsive and compatible with all modern web browsers. (5) Database tables shall follow Third Normal Form (3NF) to minimize redundancy and improve data consistency. (6) All data transactions shall be validated before processing. (7) The system shall support at least 100 concurrent users without significant performance degradation. (8) This project is being developed as an academic project and shall be fully demonstrable in a local development environment.

2.6 User Documentation

The following user documentation shall be delivered along with the LMS software: (1) User Manual – a comprehensive guide covering all system functionalities and workflows for administrators, librarians, and members. (2) Administrator Guide – detailed instructions for system setup, user management, backup procedures, and configuration settings. (3) Quick Reference Guide – a concise document describing common operations such as adding books, issuing books, and generating reports. (4) Online Help System – context-sensitive help accessible within the application. (5) Installation Guide – step-by-step instructions for setting up the application and database environment. All documentation shall be provided in PDF format and made accessible through the administrator dashboard.

2.7 Assumptions and Dependencies

The following assumptions and dependencies apply to this project: Assumptions: (1) It is assumed that all hospital staff users have basic computer literacy and access to a web-enabled device. (2) It is assumed that the hospital has a stable local area network (LAN) or internet connection available at all workstations. (3) It is assumed that patient data entered into the system is accurate and verified at the point of registration. (4) It is assumed that the MySQL database server will be available 24/7 with appropriate backup mechanisms in place. Dependencies: (1) The system depends on Node.js v18+ and npm package ecosystem for backend services. (2) The frontend is dependent on React.js v18+ and associated libraries including React Router and Axios. (3) Database operations are dependent on MySQL 8.0+ and the Sequelize ORM for query management. (4) PDF generation for bills and reports is dependent on the PDFKit or jsPDF library. (5) Authentication depends on the jsonwebtoken (JWT) npm package. (6) If email notification features are implemented, the system will depend on an SMTP service such as Nodemailer with Gmail or SendGrid.

3. External Interface Requirements

3.1 User Interfaces

The user interface shall provide separate role-appropriate dashboards for Student, Faculty, Librarian, Department Coordinator, and Administrator. Common controls include navigation, search, forms, confirmation dialogs, validation messages, notifications, profile/account controls, and logout. Student and Faculty screens

shall support book search, availability checking, borrowing, renewal, borrowing history, and requests. Librarian screens shall support book management, user management, issue/return, renewal, fines, and requests. Administrator and Department Coordinator screens shall support user, resource, report, and management functions. Error messages shall identify the problem and provide a clear corrective action.

3.2 Hardware Interfaces

The system shall support desktop/laptop computers, smartphones, and tablets through a modern web browser. Client devices shall communicate with the application through HTTPS. No specialized hardware is required for normal operation. The server environment shall provide sufficient CPU, memory, storage, and network capacity for concurrent users and database operations.

3.3 Software Interfaces

The application shall use Supabase PostgreSQL for storing users, books, borrowing records, requests, fines, reports, and related data. The web application shall be developed using Next.js 15, TypeScript, Tailwind CSS, and Lucide. Authentication shall use Supabase Auth, while Row-Level Security shall control access to data based on user roles. The application shall communicate with Supabase through secure APIs and handle unavailable services gracefully.

3.4 Communications Interfaces

All browser-to-server communication shall use HTTPS. Application requests and responses shall use standard HTTP methods and structured data such as JSON. Authentication and session information shall be protected during transmission. Supabase Realtime may be used for real-time updates where required. Network failures, timeouts, and unavailable services shall produce user-friendly error messages and shall not corrupt library records.

4. System Features

4.1 User Authentication and Role-Based Access Control

4.1.1 Description and Priority

Priority: High. Provides secure registration, login, logout, authentication, and role-based access for Student, Faculty, Librarian, Department Coordinator, and Administrator.

4.1.2 Stimulus/Response Sequences

- *User enters username/email and password and selects Login.*
- *System validates the entered credentials.*
- *If the credentials are invalid, the system displays an appropriate error message.*
- *If authentication is successful, the system creates a secure session.*
- *System redirects the user to the dashboard according to the assigned role.*
- *Unauthorized users are denied access to restricted functions.*

4.1.3 Functional Requirements

REQ-1: The system shall allow registered users to log in using valid authentication credentials.

REQ-2: The system shall validate required login fields and display an error for incomplete or invalid input.

REQ-3: The system shall provide role-based access for Student, Faculty, Librarian, Department Coordinator, and Administrator.

REQ-4: The system shall prevent unauthorized users from accessing restricted modules and functions.

REQ-5: The system shall allow authenticated users to securely log out of the system.

4.2 User and Profile Management

4.2.1 Description and Priority

Priority: High. Allows authorized users to manage user accounts, profiles, roles, and library access privileges.

4.2.2 Stimulus/Response Sequences

- *Administrator or authorized staff selects user management.*
- *System displays the available user records.*
- *Authorized user adds, updates, or views user information.*
- *System validates the entered information.*
- *Invalid or incomplete information is rejected with an appropriate message.*
- *System saves valid user details.*

4.2.3 Functional Requirements

REQ-6: The system shall allow authorized users to create and manage user accounts.

REQ-7: The system shall allow users to view and update permitted profile information.

REQ-8: The system shall assign appropriate roles to users based on authorization.

REQ-9: The system shall maintain student and faculty borrowing privileges and limits.

REQ-10: The system shall prevent unauthorized users from modifying restricted user information.

4.3 Book Management

4.3.1 Description and Priority

Priority: High. Enables authorized library staff to add, view, update, and delete book records and maintain book availability information.

4.3.2 Stimulus/Response Sequences

- *Librarian selects Book Management.*
- *System displays the available book records.*
- *Librarian enters or modifies book details.*
- *System validates the entered book information.*
- *Invalid information is rejected with an appropriate error message.*
- *System stores the valid book record and updates its status.*

4.3.3 Functional Requirements

REQ-11: The system shall allow authorized librarians to add new book records.

REQ-12: The system shall store book details including title, author, category, ISBN, publisher, edition, copies, and shelf location.

REQ-13: The system shall allow authorized users to update permitted book information.

REQ-14: The system shall allow authorized librarians to delete eligible book records.

REQ-15: The system shall maintain the availability and status of each book.

REQ-16: The system shall reject incomplete or invalid book information before saving the record.

4.4 Book Search and Catalog

4.4.1 Description and Priority

Priority: High. Allows students, faculty, and authorized users to search the library catalog and check book availability.

4.4.2 Stimulus/Response Sequences

- *User enters a title, author, category, ISBN, or keyword.*
- *System validates the search input.*
- *System retrieves matching book records.*
- *System displays book details and current availability.*
- *If no matching book is found, the system displays an appropriate message.*

4.4.3 Functional Requirements

REQ-17: The system shall allow users to search books by title, author, category, ISBN, and keyword.

REQ-18: The system shall display matching book records based on the search criteria.

REQ-19: The system shall display the current availability status of books.

REQ-20: The system shall display an appropriate message when no matching book is found.

REQ-21: The system shall provide updated availability information for books.

4.5 Book Issue Management

4.5.1 Description and Priority

Priority: High. Manages the issue of books to eligible students and faculty while checking availability and borrowing limits.

4.5.2 Stimulus/Response Sequences

- *Librarian selects a user and book for issue.*
- *System checks the user's borrowing eligibility.*
- *System checks the availability of the selected book.*
- *If the conditions are satisfied, the system creates the borrowing record.*
- *System calculates and records the applicable due date.*
- *If the book is unavailable or the user has reached the borrowing limit, the issue request is rejected.*

4.5.3 Functional Requirements

REQ-22: The system shall allow authorized librarians to issue available books to eligible users.

REQ-23: The system shall verify book availability before issuing a book.

REQ-24: The system shall verify the user's borrowing limit before issuing a book.

REQ-25: The system shall calculate and record the due date for each issued book.

REQ-26: The system shall update book availability after a successful issue transaction.

REQ-27: The system shall reject an issue transaction when the book is unavailable or the borrowing limit is exceeded.

4.6 Book Return and Renewal

4.6.1 Description and Priority

Priority: High. Manages the return and renewal of borrowed books and updates their availability.

4.6.2 Stimulus/Response Sequences

- *Librarian selects an active borrowing record.*

- *System verifies the borrowing record.*
- *User returns the book or requests renewal.*
- *System checks whether renewal is permitted.*
- *System updates the due date when renewal is successful.*
- *System updates the book availability after return.*

4.6.3 Functional Requirements

REQ-28: The system shall allow authorized librarians to record returned books.

REQ-29: The system shall update book availability after a successful return.

REQ-30: The system shall allow eligible users to renew borrowed books according to the configured renewal limit.

REQ-31: The system shall update the due date when a renewal is successfully completed.

REQ-32: The system shall reject renewal requests when the renewal limit or other configured conditions are not satisfied.

4.7 Borrowing History Management

4.7.1 Description and Priority

Priority: Medium. Maintains records of current and previous borrowing transactions for users and authorized library staff.

4.7.2 Stimulus/Response Sequences

- *User selects Borrowing History.*
- *System verifies the user's authorization.*
- *System retrieves the user's borrowing records.*
- *System displays active and completed borrowing transactions.*
- *Authorized library staff may view relevant circulation records.*

4.7.3 Functional Requirements

REQ-33: The system shall maintain a record of each book issue and return transaction.

REQ-34: The system shall allow students and faculty to view their permitted borrowing history.

REQ-35: The system shall display active borrowing records separately from completed transactions.

REQ-36: The system shall allow authorized librarians and administrators to view relevant circulation records.

REQ-37: The system shall prevent unauthorized users from viewing another user's borrowing history.

4.8 Faculty Book Request

4.8.1 Description and Priority

Priority: Medium. Allows faculty members to submit book requests and enables authorized library staff to manage those requests.

4.8.2 Stimulus/Response Sequences

- *Faculty member enters the required book request details.*
- *System validates the request information.*
- *System creates the request with an appropriate status.*
- *Librarian reviews the request.*
- *System updates the request status based on the librarian's action.*
- *Faculty member can view the status of the submitted request.*

4.8.3 Functional Requirements

REQ-38: The system shall allow faculty members to submit book requests.

REQ-39: The system shall validate required information before creating a book request.

REQ-40: The system shall maintain the status of each faculty book request.

REQ-41: The system shall allow authorized librarians to view and manage faculty book requests.

REQ-42: The system shall allow faculty members to view the status of their submitted requests.

4.9 Department Resource Management

4.9.1 Description and Priority

Priority: Medium. Allows authorized users to manage and monitor library resources associated with departments.

4.9.2 Stimulus/Response Sequences

- *Department Coordinator selects resource management.*
- *System displays the available department resources.*
- *Coordinator adds or updates resource information.*
- *System validates the entered information.*
- *Valid information is stored and displayed.*
- *Invalid information is rejected with an appropriate error message.*

4.9.3 Functional Requirements

REQ-43: The system shall allow authorized department coordinators to manage department resource information.

REQ-44: The system shall allow authorized users to add and update permitted resource records.

REQ-45: The system shall maintain the status and relevant details of department resources.

REQ-46: The system shall prevent unauthorized users from modifying department resource records.

4.10 Fine and Overdue Monitoring

4.10.1 Description and Priority

Priority: High. Monitors overdue books and manages applicable fine information based on the configured library rules.

4.10.2 Stimulus/Response Sequences

- *System checks borrowing records against their due dates.*
- *System identifies overdue books.*
- *System calculates the applicable fine according to configured rules.*
- *Authorized staff can view and manage fine information.*
- *The system updates the fine status when the required action is completed.*

4.10.3 Functional Requirements

REQ-47: The system shall identify borrowing records that have exceeded their due dates.

REQ-48: The system shall calculate applicable fines according to configured library rules.

REQ-49: The system shall maintain fine information associated with overdue borrowing records.

REQ-50: The system shall allow authorized users to view overdue and fine records.

REQ-51: The system shall prevent unauthorized users from modifying fine information.

4.11 Report And Analysis

4.11.1 Description and Priority

Priority: High. Provides authorized users with reports related to books, users, borrowing activities, overdue records, fines, and library operations.

4.11.2 Stimulus/Response Sequences

- *Authorized user selects the required report type.*
- *System retrieves the relevant library data.*
- *System validates the requested report parameters.*
- *System generates the report.*
- *If the required data is unavailable, the system displays an appropriate message.*

4.11.3 Functional Requirements

REQ-52: The system shall allow authorized users to generate library management reports.

REQ-53: The system shall generate reports related to books, users, borrowing, returns, overdue records, and fines.

REQ-54: The system shall allow reports to be generated using relevant date or category filters.

REQ-55: The system shall provide generated reports in supported formats such as PDF or Excel.

REQ-56: The system shall restrict report generation and access to authorized users.

REQ-57: The system shall display an appropriate message when insufficient data is available for a requested report.

5. Other Nonfunctional Requirements

5.1 Performance Requirements

- *The system shall process user login requests and display the appropriate dashboard within 3 seconds under normal operating conditions.*
- *The book search function shall display matching book records within 3 seconds for normal database loads.*
- *The system shall update book availability immediately after successful issue and return transactions.*
- *The report generation function shall generate standard library reports within 10 seconds under normal operating conditions.*
- *The system shall support multiple users accessing the application simultaneously without significant performance degradation.*

5.2 Safety Requirements

- *The system shall validate all user inputs before storing or processing them to prevent invalid library records.*
- *Confirmation messages shall be displayed before permanently deleting user, book, or other important records.*
- *The system shall prevent a book from being issued when no available copy exists.*
- *The system shall prevent users from exceeding their configured borrowing limits.*
- *The system shall maintain database consistency during issue, return, renewal, and fine transactions to prevent incorrect library records.*

5.3 Security Requirements

- *The system shall use secure authentication mechanisms to verify users before providing access to protected functions.*
- *The system shall enforce role-based access for Student, Faculty, Librarian, Department Coordinator, and Administrator accounts.*
- *The application shall use HTTPS for communication between users and the deployed application.*
- *Supabase Authentication shall be used to securely manage user authentication and sessions.*
- *Row-Level Security (RLS) shall be implemented to restrict access to database records according to user roles and permissions.*
- *The system shall validate and sanitize user inputs to reduce the risk of SQL Injection, Cross-Site Scripting (XSS), and other common attacks.*
- *Administrative and other restricted functions shall only be accessible to authorized users.*

5.4 Software Quality Attributes

- **Usability:** *The system shall provide simple, consistent, and role-specific interfaces so that users can perform common library operations with minimal navigation.*
- **Maintainability:** *The application shall use modular components and organized code to simplify debugging, maintenance, and future modifications.*
- **Reliability:** *The system shall maintain accurate book, user, borrowing, return, renewal, and fine records during normal operation.*
- **Scalability:** *The system shall support increasing numbers of users, books, and borrowing transactions without major changes to the application architecture.*
- **Portability:** *The web application shall be accessible through modern web browsers on desktops, laptops, tablets, and smartphones.*
- **Testability:** *Individual modules such as authentication, book management, issue, return, renewal, and report generation shall be independently testable.*

5.5 Business Rules

- *Only authenticated users with appropriate roles shall be allowed to access protected library functions.*
- *Students and Faculty shall be allowed to borrow books according to their configured borrowing limits.*
- *Only available book copies shall be issued to eligible users.*
- *Only authorized Librarians shall perform book issue and return operations.*
- *Faculty members shall be allowed to submit book requests, while authorized Librarians shall manage those requests.*
- *Department Coordinators shall be allowed to manage permitted department resource information.*
- *Fine amounts shall be calculated according to the configured library rules for overdue books.*
- *Students and Faculty shall only be allowed to view their permitted personal borrowing history.*
- *Administrator functions shall only be accessible to authorized Administrator accounts.*

6. Other Requirements

The application components—specifically the authentication module, role-based access control, book management components, borrowing and return services, and report generation modules—shall be designed using modular and reusable components to facilitate easy maintenance and integration into future library management systems.

Appendix A: Glossary

- **Student:** An authorized library user who can search books, borrow and renew books, and view permitted borrowing history.
- **Faculty:** An authorized academic user who can search, borrow, renew books, view borrowing history, and submit book requests.
- **Librarian:** An authorized user responsible for managing books, users, book issue and return transactions, requests, fines, and library records.
- **Department Coordinator:** An authorized user responsible for managing permitted department-related library resources.
- **Administrator:** An authorized user with higher-level access to manage users, system records, reports, and administrative functions.
- **Book Record:** A database record containing information such as title, author, category, ISBN, publisher, edition, copies, shelf location, and availability status.
- **Borrowing Record:** A record containing information about a book issued to a user, including issue date, due date, return date, and borrowing status.
- **Book Request:** A request submitted by a Faculty member for a required library book.
- **Fine:** A monetary amount calculated according to library rules when a borrowed book is returned after its due date.
- **Role-Based Access Control (RBAC):** A security mechanism that provides system permissions based on the user's assigned role.
- **Supabase:** The backend platform used by the system for PostgreSQL database, authentication, Row-Level Security, and related services.
- **Row-Level Security (RLS):** A database security mechanism used to control which records users can access or modify.
- **Next.js:** The web application framework used to develop the Library Management System.

Appendix B: Analysis Models

1. UML Class Diagram

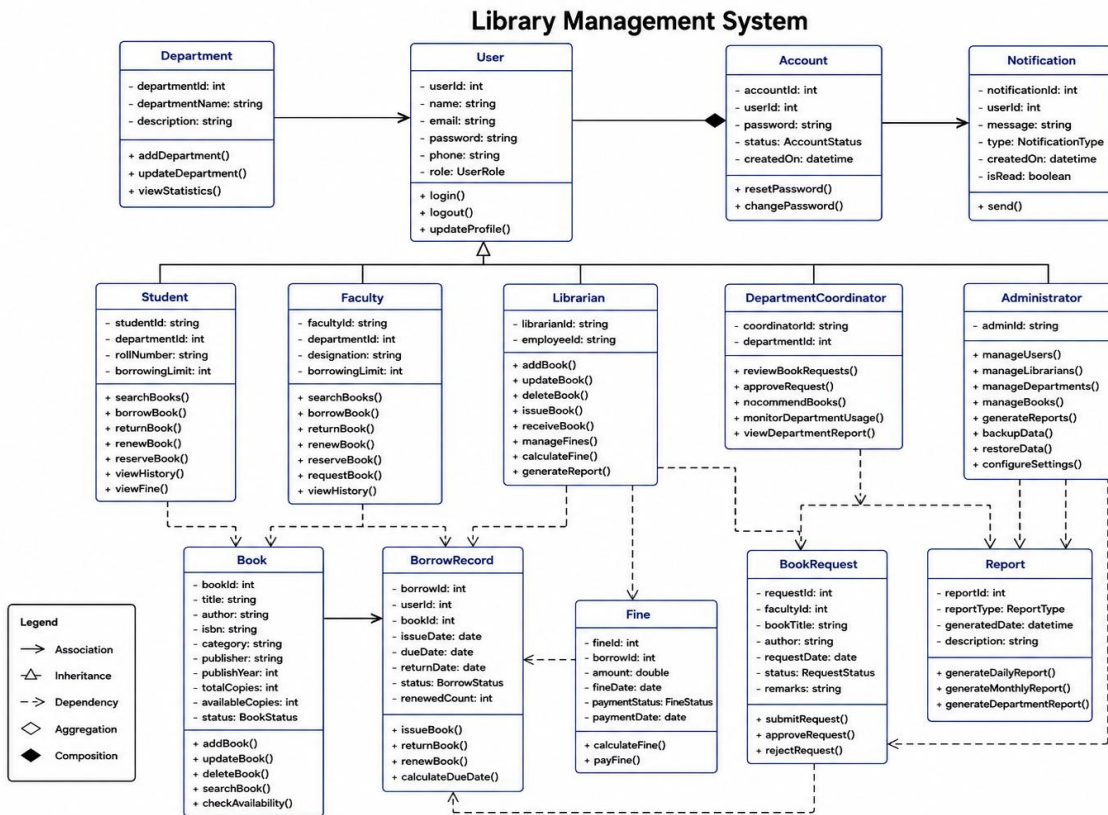


Figure B.1: UML Class Diagram – Library Management System

2. UML Activity Diagram

2.0 LIBRARY USER MANAGEMENT

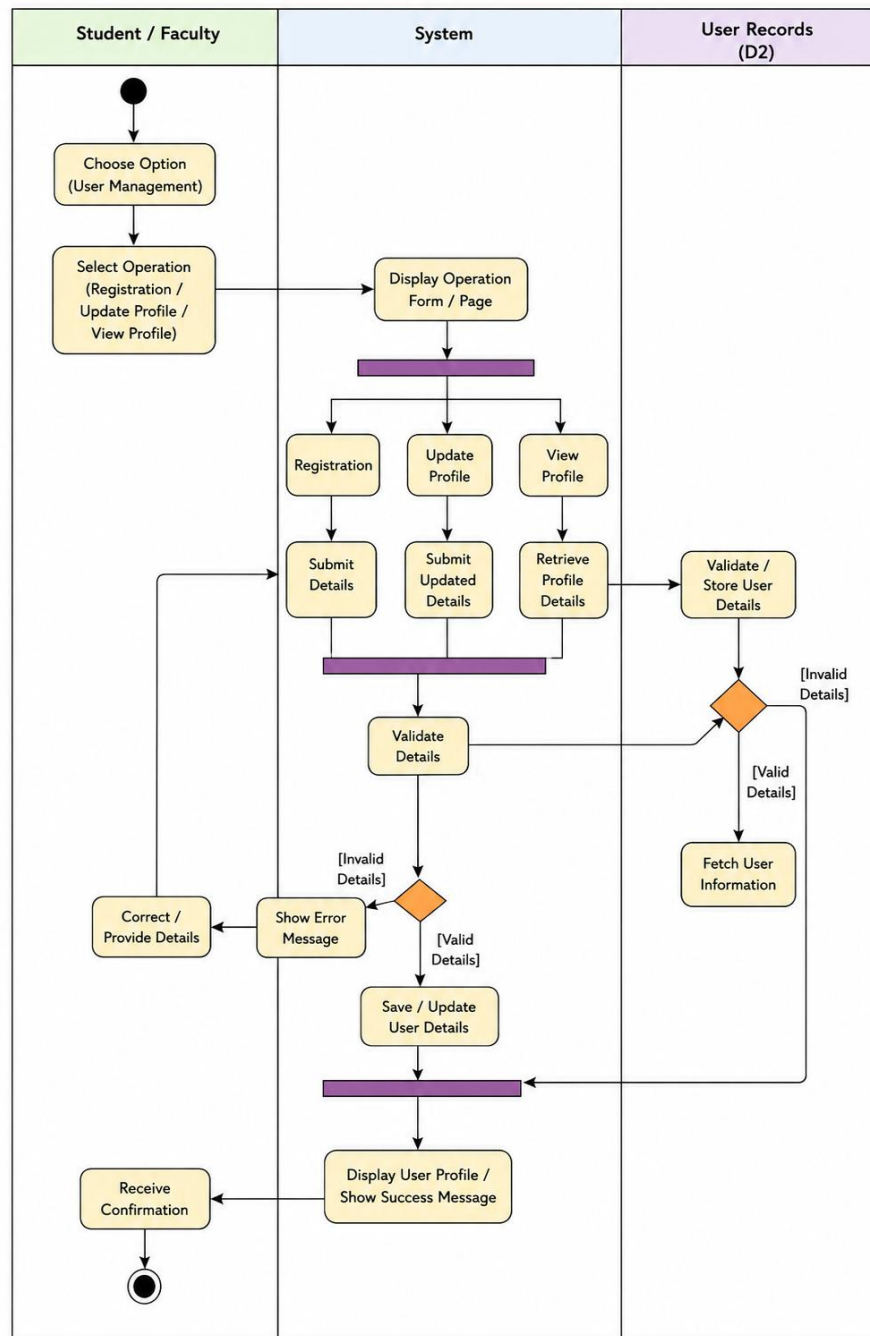


Figure B.2: UML Activity Diagram – 2.0 Library User Management

3.0 BOOK MANAGEMENT

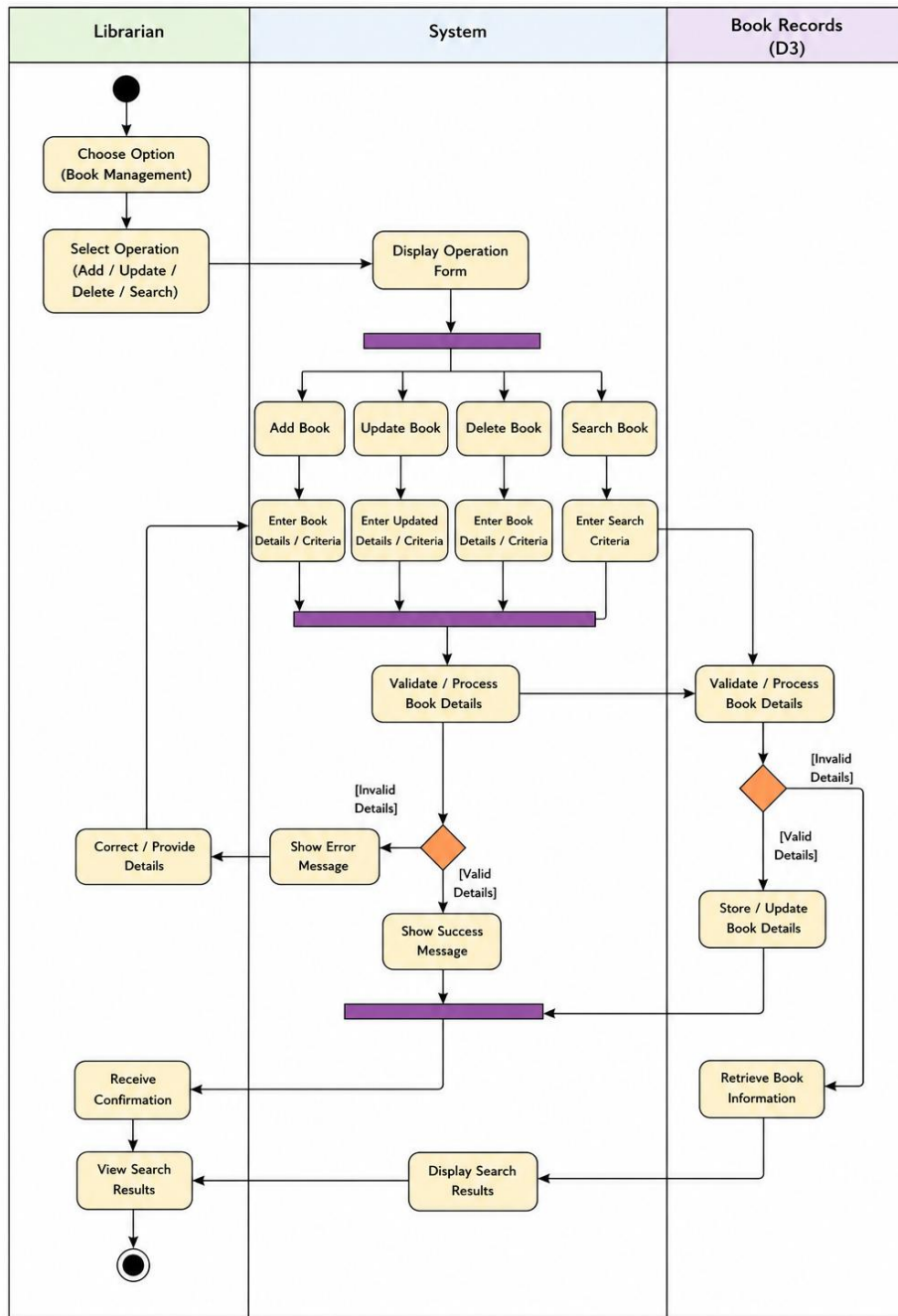


Figure B.3: UML Activity Diagram – 3.0 Book Management

3. UML Sequence Diagram

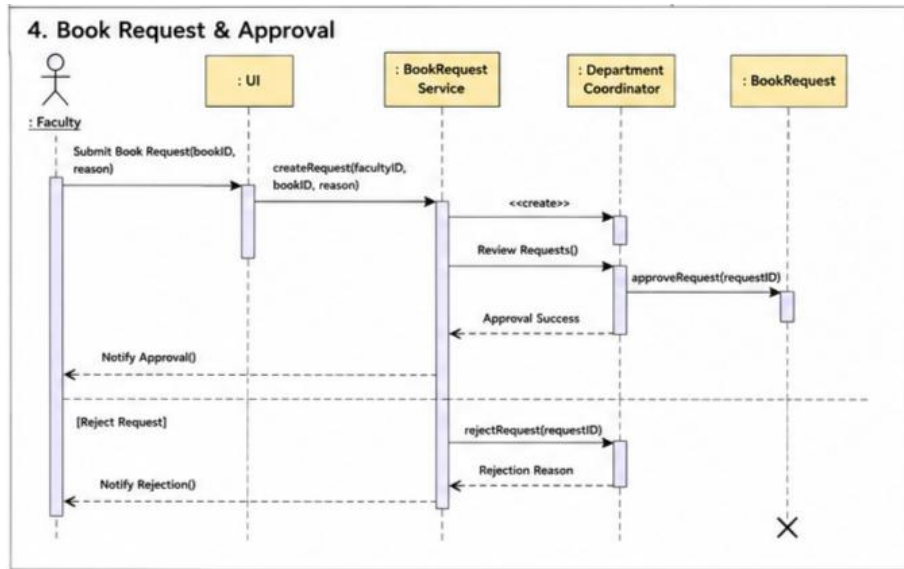


Figure B.4: UML Sequence Diagram – Book Request & Approval

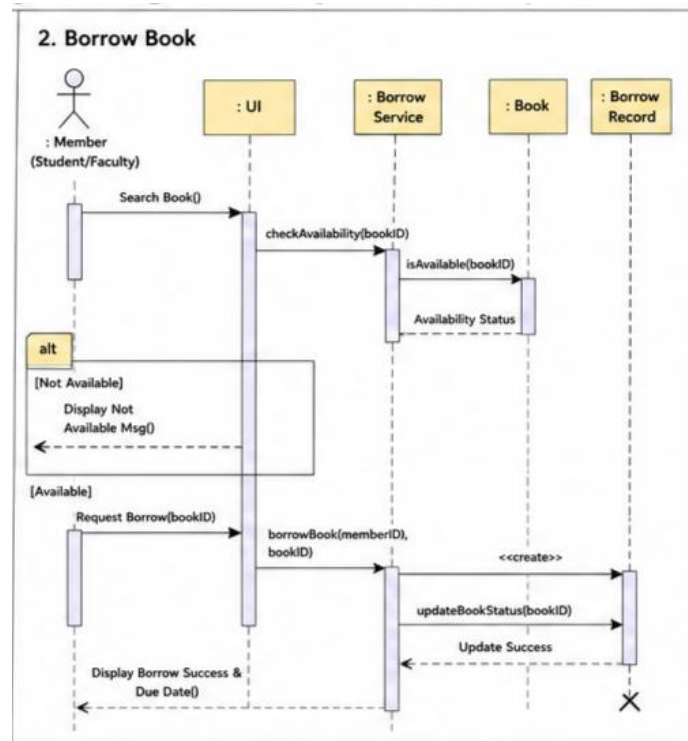


Figure B.5: UML Sequence Diagram – Borrow Book

4. State Chart Diagram

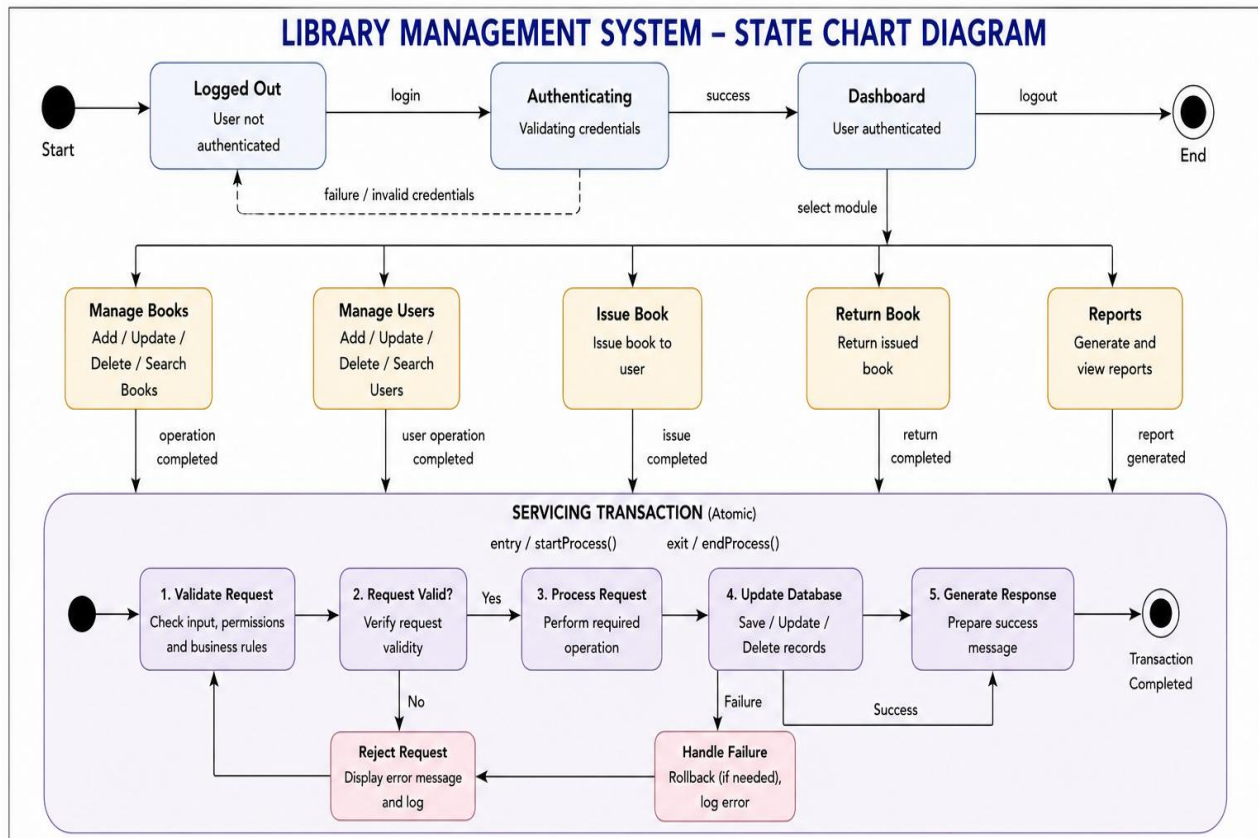


Figure B.6: UML State Chart Diagram – Library Management System

5. UML Component

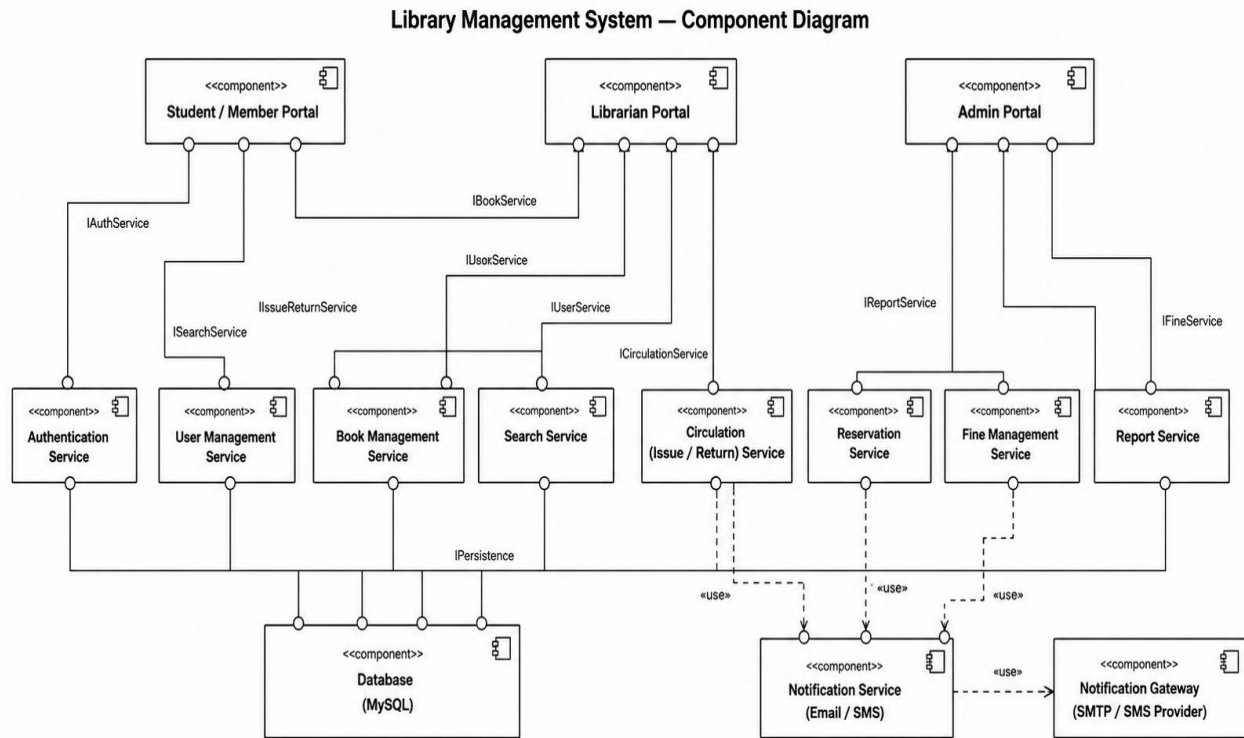


Figure B.7: UML Component Diagram – Library Management System

6. UML Deployment

DEPLOYMENT DIAGRAM – LIBRARY MANAGEMENT SYSTEM

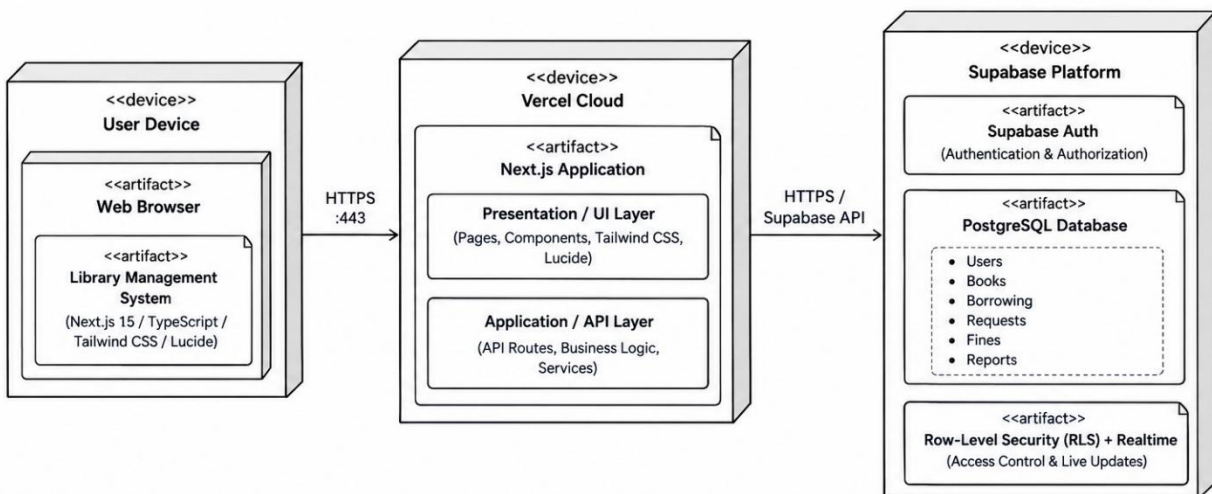


Figure B.8: UML Deployment Diagram – Library Management System

Source Code**Book Catalog (page.tsx)**

```

"use client";

import React, { useState, useEffect } from "react";
import { useAuth } from "@context/AuthContext";
import { DataStore } from "@lib/data-store";
import { Book } from "@lib/types";
import {
  Search,
  Plus,
  BookOpen,
  Filter,
  Edit2,
  Trash2,
  Eye,
  CheckCircle2,
  XCircle,
  Sparkles,
} from "lucide-react";

export default function BookCatalogPage() {
  const { canManageBooks } = useAuth();
  const [books, setBooks] = useState<Book[]>([]);
  const [searchQuery, setSearchQuery] = useState("");
  const [categoryFilter, setCategoryFilter] =
    useState("ALL");
  const [statusFilter, setStatusFilter] = useState("ALL");
  const [quickFilter, setQuickFilter] = useState("");

  const [selectedBook, setSelectedBook] = useState<Book |
    null>(null);
  const [viewModalOpen, setViewModalOpen] = useState(false);
  const [addModalOpen, setAddModalOpen] = useState(false);

  const [formData, setFormData] = useState({
    title: "",
    author: "",
    isbn: "",
    category: "Computer Science",
    publisher: "",
    edition: "1st Edition",
    total_copies: 5,
    shelf_location: "Shelf CS-01",
    description: "",
  });

  const loadBooks = () => {
    setBooks(DataStore.getBooks());
  };

  useEffect(() => {
    loadBooks();
    const handleDataChange = () => loadBooks();
    window.addEventListener("lms_data_change",
      handleDataChange);
    return () =>
      window.removeEventListener("lms_data_change",
        handleDataChange);
  }, []);

  const handleResetFilters = () => {
    setSearchQuery("");
    setCategoryFilter("ALL");
    setStatusFilter("ALL");
    setQuickFilter("");
  };

  const handleAddSubmit = (e: React.FormEvent) => {
    e.preventDefault();
    if (!formData.title || !formData.author
      || !formData.isbn) {
      alert("Title, Author, and ISBN are required.");
      return;
    }

    DataStore.saveBook({
      isbn: formData.isbn,
      title: formData.title,
      author: formData.author,
      category: formData.category,
      publisher: formData.publisher || "Academic Press",
      edition: formData.edition || "1st Edition",
      total_copies: Number(formData.total_copies),
      available_copies: Number(formData.total_copies),
      shelf_location: formData.shelf_location || "Shelf A-
01",
      description: formData.description,
      is_active: true,
    });

    setAddModalOpen(false);
    setFormData({
      title: "",
      author: "",
      isbn: "",
      category: "Computer Science",
      publisher: "",
      edition: "1st Edition",
      total_copies: 5,
      shelf_location: "Shelf CS-01",
      description: "",
    });
    loadBooks();
  };

  const categories = Array.from(new Set(books.map((b) =>
    b.category)));

  const filteredBooks = books.filter((b) => {
    const query = (searchQuery ||
      quickFilter).toLowerCase();
    const matchesSearch =
      b.title.toLowerCase().includes(query) ||
      b.author.toLowerCase().includes(query) ||
      b.isbn.toLowerCase().includes(query);
    const matchesCat = categoryFilter === "ALL" ||
      b.category === categoryFilter;
    const matchesStatus =
      statusFilter === "ALL" ||
      (statusFilter === "AVAILABLE" && b.available_copies >
        0) ||
      (statusFilter === "CHECKED_OUT" && b.available_copies
        === 0);

    return matchesSearch && matchesCat && matchesStatus;
  });

  return (
    <div className="space-y-5">
      { /* Top Filter Bar */ }
      <div className="bg-white p-4 rounded-lg border border-
        slate-200 shadow-sm flex flex-col md:flex-row gap-3 items-
        center justify-between">
        <div className="flex flex-1 flex-col sm:flex-row
          gap-2.5 w-full">
          <div className="relative flex-1">
            <input
              type="text"
              placeholder="Search by title, author, ISBN..."
              value={searchQuery}
              onChange={(e) =>
                setSearchQuery(e.target.value)}
              className="w-full px-3 py-2 rounded border
                border-slate-300 text-slate-900 text-xs focus:outline-none
                focus:border-blue-600 bg-white"
            />
          </div>
          <select
            value={categoryFilter}
            onChange={(e) =>
              setCategoryFilter(e.target.value)}
          >

```

Page 24

```

<div className="overflow-x-auto">
  <table className="w-full text-left border-collapse
text-xs">
    <thead>
      <tr className="bg-slate-50 text-slate-500
uppercase font-semibold border-b border-slate-200">
        <th className="p-3">ISBN</th>
        <th className="p-3">BOOK DETAILS</th>
        <th className="p-3">CATEGORY</th>
        <th className="p-3">PUBLISHER & YEAR</th>
        <th className="p-3">AVAILABILITY</th>
        <th className="p-3">SHELF</th>
        <th className="p-3">STATUS</th>
        <th className="p-3 text-right">ACTIONS</th>
      </tr>
    </thead>
    <tbody className="divide-y divide-slate-100
text-slate-700">
      {filteredBooks.length === 0 ? (
        <tr>
          <td colspan={8} className="p-8 text-center
text-slate-400">
            No book records found matching your
filters.
          </td>
        </tr>
      ) : (
        filteredBooks.map((book) => (
          <tr key={book.id} className="hover:bg-
slate-50/80 transition-colors">
            <td className="p-3 font-mono text-slate-
500">{book.isbn}</td>
            <td className="p-3">
              <div className="font-bold text-slate-
900 text-xs">{book.title}</div>
              <div className="text-[11px] text-
slate-500">by {book.author}</div>
            </td>
            <td className="p-3">
              <span className="px-2 py-0.5 rounded
bg-slate-100 text-slate-700 text-xs font-medium">
                {book.category}
              </span>
            </td>
            <td className="p-3 text-slate-600">
              <div>{book.publisher} ||
"Pearson"</div>
            <div className="text-[11px] text-
slate-400">{book.edition} || "1st Edition"</div>
            </td>
            <td className="p-3">
              <span className="px-2.5 py-0.5
rounded-full bg-emerald-50 text-emerald-700 border border-
emerald-200 font-bold text-[11px]">
                {book.available_copies} /
{book.total_copies}
              </span>
            </td>
            <td className="p-3 text-slate-500 font-
medium">{book.shelf_location} || "Shelf A-01"</td>
            <td className="p-3">
              {book.available_copies > 0 ? (
                <span className="px-2 py-0.5 rounded
bg-emerald-50 text-emerald-700 border border-emerald-200
text-xs font-bold uppercase">
                  AVAILABLE
                </span>
              ) : (
                <span className="px-2 py-0.5 rounded
bg-amber-50 text-amber-700 border border-amber-200 text-xs
font-bold uppercase">
                  CHECKED OUT
                </span>
              )
            </td>
            <td className="p-3 text-right">
              <button
                onClick={() => {
                  setSelectedBook(book);

```

```

        type="text"
        required
        value={formData.title}
        onChange={(e) =>
setFormData({ ...formData, title: e.target.value })}
        className="w-full px-3 py-1.5 rounded
border border-slate-300 text-slate-900 focus:border-blue-600"
    />
</div>

    <div>
        <label className="block font-semibold text-
slate-700 mb-1">Author *</label>
        <input
            type="text"
            required
            value={formData.author}
            onChange={(e) =>
setFormData({ ...formData, author: e.target.value })}
            className="w-full px-3 py-1.5 rounded
border border-slate-300 text-slate-900 focus:border-blue-600"
        />
    </div>

    <div className="grid grid-cols-2 gap-3">
        <div>
            <label className="block font-semibold
text-slate-700 mb-1">ISBN *</label>
            <input
                type="text"
                required
                value={formData.isbn}
                onChange={(e) =>
setFormData({ ...formData, isbn: e.target.value })}
                className="w-full px-3 py-1.5 rounded
border border-slate-300 text-slate-900 focus:border-blue-600"
            />
        </div>
        <div>
            <label className="block font-semibold
text-slate-700 mb-1">Total Copies</label>
            <input
                type="number"
                min="1"
                required
                value={formData.total_copies}
                onChange={(e) =>
setFormData({ ...formData, total_copies:
parseInt(e.target.value) || 1 })}
                className="w-full px-3 py-1.5 rounded
border border-slate-300 text-slate-900 focus:border-blue-600"
            />
        </div>
    </div>

    <div className="grid grid-cols-2 gap-3">
        <div>
            <label className="block font-semibold
text-slate-700 mb-1">Category</label>
            <input
                type="text"
                value={formData.category}
                onChange={(e) =>
setFormData({ ...formData, category: e.target.value })}
                className="w-full px-3 py-1.5 rounded
border border-slate-300 text-slate-900 focus:border-blue-600"
            />
        </div>
        <div>
            <label className="block font-semibold
text-slate-700 mb-1">Shelf Location</label>
            <input
                type="text"
                value={formData.shelf_location}

```

```

      onChange={(e) =>
setFormData({ ...formData, shelf_location:
e.target.value })}
      className="w-full px-3 py-1.5 rounded
border border-slate-300 text-slate-900 focus:border-blue-
600"
    />
  </div>
</div>

  <div className="flex justify-end gap-2 pt-2
border-t">
    <button
      type="button"
      onClick={() => setAddModalOpen(false)}
      className="px-3 py-1.5 rounded border
border-slate-300 text-slate-700 hover:bg-slate-50 font-
semibold"
    >
      Cancel
    </button>
    <button
      type="submit"
      className="px-4 py-1.5 rounded bg-blue-600
hover:bg-blue-700 text-white font-semibold shadow-sm"
    >
      Save Book
    </button>
  </div>
</form>
</div>
</div>
  )}
</div>
);
}

```

Member Directory (page.tsx)

```

"use client";

import React, { useState, useEffect } from "react";
import { DataStore } from "@/lib/data-store";
import { UserProfile, UserRole, UserStatus } from
"@/lib/types";
import { useAuth } from "@/context/AuthContext";
import {
  Users,
  Search,
  Plus,
  ShieldCheck,
  Building2,
  Edit,
  Trash2,
  UserCheck,
  UserX,
  Phone,
  Mail,
  X,
} from "lucide-react";

export default function MemberDirectoryPage() {
  const { canManageMembers } = useAuth();
  const [members, setMembers] = useState<UserProfile[]>([]);
  const [searchQuery, setSearchQuery] = useState("");
  const [roleFilter, setRoleFilter] = useState("ALL");
  const [isModalOpen, setIsModalOpen] = useState(false);
  const [editingMemberId, setEditingMemberId] =
useState<string | null>(null);

  const [formData, setFormData] = useState({
    username: "",
    password: ".....",
    full_name: "",
    email: "",
    role: "STUDENT" as UserRole,
    department: "Computer Science & Engineering",
    phone: "",

```

```

    max_books_allowed: 3,
    status: "ACTIVE" as UserStatus,
  });

  const loadMembers = () => {
    setMembers(DataStore.getProfiles());
  };

  useEffect(() => {
    loadMembers();
    const handleDataChange = () => loadMembers();
    window.addEventListener("lms_data_change",
handleDataChange);
    return () =>
window.removeEventListener("lms_data_change",
handleDataChange);
  }, []);

  const handleOpenCreateModal = () => {
    setEditingMemberId(null);
    setFormData({
      username: "",
      password: ".....",
      full_name: "",
      email: "",
      role: "STUDENT",
      department: "Computer Science & Engineering",
      phone: "+1 555-0199",
      max_books_allowed: 3,
      status: "ACTIVE",
    });
    setIsModalOpen(true);
  };

  const handleOpenEditModal = (member: UserProfile) => {
    setEditingMemberId(member.id);
    setFormData({
      username: member.full_name.split("
")[0].toLowerCase(),
      password: ".....",
      full_name: member.full_name,
      email: member.email,
      role: member.role,
      department: member.department || "Computer Science",
      phone: member.phone || "+1 555-0199",
      max_books_allowed: member.max_books_allowed,
      status: member.status || "ACTIVE",
    });
    setIsModalOpen(true);
  };

  const handleFormSubmit = (e: React.FormEvent) => {
    e.preventDefault();
    if (!formData.full_name || !formData.email) {
      alert("Full Name and Email Address are required.");
      return;
    }

    DataStore.saveProfile({
      id: editingMemberId || undefined,
      full_name: formData.full_name,
      email: formData.email,
      role: formData.role,
      department: formData.department,
      phone: formData.phone || "+1 555-0199",
      max_books_allowed: Number(formData.max_books_allowed)
|| 3,
      status: formData.status,
    });

    setIsModalOpen(false);
    loadMembers();
  };

  const handleDeleteMember = (member: UserProfile) => {
    if (confirm("Are you sure you want to delete
${member.full_name}?")) {
      DataStore.deleteProfile(member.id);
      loadMembers();
    }
  };

```

```

    }
  };

  const handleToggleStatus = (member: UserProfile) => {
    const newStatus = member.status === "ACTIVE" ?
    "SUSPENDED" : "ACTIVE";
    DataStore.saveProfile({
      ...member,
      status: newStatus,
    });
    loadMembers();
  };

  const handleRoleChange = (role: UserRole) => {
    let limit = 3;
    if (role === "FACULTY" || role === "COORDINATOR") limit
    = 5;
    if (role === "ADMIN" || role === "LIBRARIAN") limit =
    99;
    setFormData((prev) => ({ ...prev, role,
    max_books_allowed: limit }));
  };

  const filteredMembers = members.filter((m) => {
    const matchesSearch =

m.full_name.toLowerCase().includes(searchQuery.toLowerCase()
) ||

m.email.toLowerCase().includes(searchQuery.toLowerCase()) ||
    (m.department &&
    m.department.toLowerCase().includes(searchQuery.toLowerCase(
    )));
    const matchesRole = roleFilter === "ALL" || m.role ===
    roleFilter;
    return matchesSearch && matchesRole;
  });

  return (
    <div className="space-y-5">
      { /* Top Filter Bar */ }
      <div className="bg-white p-4 rounded-lg border border-
      slate-200 shadow-sm flex flex-col md:flex-row gap-3 items-
      center justify-between">
        <div className="flex flex-1 flex-col sm:flex-row
        gap-2.5 w-full">
          <div className="relative flex-1">
            <input
              type="text"
              placeholder="Search member name, email,
              department..."
              value={searchQuery}
              onChange={(e) =>
                setSearchQuery(e.target.value)}
              className="w-full px-3 py-2 rounded border border-
              slate-300 text-slate-900 text-xs focus:outline-none
              focus:border-blue-600 bg-white"
            />
          </div>

          <select
            value={roleFilter}
            onChange={(e) => setRoleFilter(e.target.value)}
            className="px-3 py-2 rounded border border-
            slate-300 text-slate-700 text-xs focus:outline-none
            focus:border-blue-600 bg-white"
          >
            <option value="ALL">All Roles</option>
            <option value="STUDENT">Student</option>
            <option value="FACULTY">Faculty</option>
            <option value="LIBRARIAN">Librarian</option>
            <option value="ADMIN">Admin</option>
            <option value="COORDINATOR">Coordinator</option>
          </select>
        </div>

        {canManageMembers && (
          <button
            onClick={handleOpenCreateModal}

```

```

          className="px-3.5 py-2 rounded bg-[#2563eb]
          hover:bg-[#1d4ed8] text-white text-xs font-semibold flex
          items-center gap-1.5 shadow-sm transition-all ml-auto md:ml-
          0"
        >
          <Plus className="w-3.5 h-3.5" />
          <span>Add Member</span>
        </button>
      </div>

      { /* Members Directory Grid */ }
      <div className="grid grid-cols-1 sm:grid-cols-2
      lg:grid-cols-3 gap-4">
        {filteredMembers.map((member) => (
          <div key={member.id} className="bg-white rounded-
          lg border border-slate-200 p-5 shadow-sm space-y-3 relative
          group">
            <div className="flex items-start justify-
            between">
              <div className="flex items-center gap-3">
                <div className="w-10 h-10 rounded-full bg-
                slate-100 border border-slate-200 flex items-center justify-
                center font-bold text-slate-700 text-sm">
                  {member.full_name.charAt(0)}
                </div>
                <div>
                  <h3 className="font-bold text-slate-900
                  text-sm flex items-center gap-1.5">
                    {member.full_name}
                  </h3>
                  <p className="text-xs text-slate-
                  500">{member.email}</p>
                </div>
              </div>
              <span className={`px-2 py-0.5 rounded text-
              [10px] font-bold uppercase ${
                member.role === "ADMIN" ? "bg-purple-50
                text-purple-700 border border-purple-200" :
                member.role === "LIBRARIAN" ? "bg-amber-50
                text-amber-700 border border-amber-200" :
                member.role === "FACULTY" ? "bg-indigo-50
                text-indigo-700 border border-indigo-200" :
                "bg-blue-50 text-blue-700 border border-
                blue-200"
              }`}>
                {member.role}
              </span>
            </div>

            <div className="pt-2 border-t border-slate-100
            text-xs space-y-1.5 text-slate-600">
              <p className="flex items-center gap-1.5">
                <Building2 className="w-3.5 h-3.5 text-
                slate-400" />
                <span>{member.department || "General
                Department"}</span>
              </p>
              <p className="flex items-center gap-1.5">
                <ShieldCheck className="w-3.5 h-3.5 text-
                slate-400" />
                <span>Max Books Limit: <strong
                className="text-slate-
                800">{member.max_books_allowed}</strong></span>
              </p>
              <p className="flex items-center justify-
              between pt-1">
                <span className={`text-[10px] font-bold px-2
                py-0.5 rounded ${
                  member.status === "SUSPENDED" ? "bg-red-50
                  text-red-600 border border-red-200" : "bg-emerald-50 text-
                  emerald-700 border border-emerald-200"
                }`}>
                  {member.status || "ACTIVE"}
                </span>

                {canManageMembers && (
                  <div className="flex items-center gap-1
                  opacity-90 group-hover:opacity-100 transition-opacity">

```

```

        value={formData.username}
        onChange={(e) =>
setFormData({ ...formData, username: e.target.value })}
            className="w-full px-3 py-2 bg-[#eef4ff]
border border-slate-300 rounded text-slate-800
focus:outline-none focus:border-blue-600 text-xs font-
medium"

                placeholder="Iyyappan"
            />
        </div>

        {/* Password */}
        <div>
            <label className="block font-semibold
text-slate-700 mb-1.5">
                Password *
            </label>
            <input
                type="password"
                required
                value={formData.password}
                onChange={(e) =>
setFormData({ ...formData, password: e.target.value })}
                className="w-full px-3 py-2 bg-[#eef4ff]
border border-slate-300 rounded text-slate-800
focus:outline-none focus:border-blue-600 text-xs"
                placeholder="....."
            />
        </div>

        {/* Full Name */}
        <div>
            <label className="block font-semibold
text-slate-700 mb-1.5">
                Full Name *
            </label>
            <input
                type="text"
                required
                value={formData.full_name}
                onChange={(e) =>
setFormData({ ...formData, full_name: e.target.value })}
                className="w-full px-3 py-2 bg-white
border border-slate-300 rounded text-slate-900
focus:outline-none focus:border-blue-600 text-xs"
                placeholder="e.g. John Doe"
            />
        </div>

        {/* Email Address */}
        <div>
            <label className="block font-semibold
text-slate-700 mb-1.5">
                Email Address *
            </label>
            <input
                type="email"
                required
                value={formData.email}
                onChange={(e) =>
setFormData({ ...formData, email: e.target.value })}
                className="w-full px-3 py-2 bg-white
border border-slate-300 rounded text-slate-900
focus:outline-none focus:border-blue-600 text-xs"
                placeholder="e.g. jdoe@university.edu"
            />
        </div>

        {/* Role */}
        <div>
            <label className="block font-semibold
text-slate-700 mb-1.5">
                Role *
            </label>
            <select
                value={formData.role}
                onChange={(e) =>
handleRoleChange(e.target.value as UserRole)}

```

```

        className="w-full px-3 py-2 bg-white
border border-slate-300 rounded text-slate-900
focus:outline-none focus:border-blue-600 text-xs"
      >
        <option value="STUDENT">Student</option>
        <option value="FACULTY">Faculty</option>
        <option
value="LIBRARIAN">Librarian</option>
        <option value="ADMIN">Admin</option>
        <option
value="COORDINATOR">Coordinator</option>
      </select>
    </div>

    {/ * Department */}
    <div>
      <label className="block font-semibold
text-slate-700 mb-1.5">
        Department
      </label>
      <input
        type="text"
        value={formData.department}
        onChange={(e) =>
          setFormData({ ...formData, department: e.target.value })}
        className="w-full px-3 py-2 bg-white
border border-slate-300 rounded text-slate-900
focus:outline-none focus:border-blue-600 text-xs"
        placeholder="e.g. Computer Science"
      />
    </div>

    {/ * Phone Number */}
    <div>
      <label className="block font-semibold
text-slate-700 mb-1.5">
        Phone Number
      </label>
      <input
        type="text"
        value={formData.phone}
        onChange={(e) =>
          setFormData({ ...formData, phone: e.target.value })}
        className="w-full px-3 py-2 bg-white
border border-slate-300 rounded text-slate-900
focus:outline-none focus:border-blue-600 text-xs"
        placeholder="e.g. +1 555-0199"
      />
    </div>

    {/ * Max Borrowing Limit */}
    <div>
      <label className="block font-semibold
text-slate-700 mb-1.5">
        Max Borrowing Limit
      </label>
      <input
        type="number"
        min="1"
        max="99"
        value={formData.max_books_allowed}
        onChange={(e) =>
          setFormData({ ...formData, max_books_allowed:
Number(e.target.value) })}
        className="w-full px-3 py-2 bg-white
border border-slate-300 rounded text-slate-900
focus:outline-none focus:border-blue-600 text-xs"
        placeholder="3"
      />
    </div>

    {/ * Action Buttons at Bottom Right */}
    <div className="flex items-center justify-end
gap-3 pt-4 border-t border-slate-200">
      <button
        type="button"
        onClick={() => setIsModalOpen(false)}

```

```

        className="px-4 py-2 border border-slate-
300 rounded text-xs font-semibold text-slate-700 hover:bg-
slate-50 transition-all"
      >
        Cancel
      </button>
      <button
        type="submit"
        className="px-5 py-2 bg-[#2563eb]
hover:bg-[#1d4ed8] text-white text-xs font-semibold rounded
shadow-sm transition-all"
      >
        {editingMemberId ? "Save Changes" :
"Create User"}
      </button>
    </div>
  </form>
</div>
</div>
  )}
</div>
);
}

```

Circulation (page.tsx)

```

"use client";

import React, { useState, useEffect } from "react";
import Link from "next/link";
import { useAuth } from "@context/AuthContext";
import { DataStore } from "@lib/data-store";
import { Book, BorrowRecord, UserProfile } from
"@lib/types";
import {
  ArrowRightLeft,
  CheckCircle2,
  Clock,
  RotateCcw,
  Search,
  Plus,
  BookOpen,
  UserCheck,
} from "lucide-react";

export default function CirculationPage() {
  const { currentUser, isStudent, isFaculty,
canIssueReturn } = useAuth();
  const [borrows, setBorrows] =
useState<BorrowRecord[]>([]);
  const [books, setBooks] = useState<Book[]>([]);
  const [members, setMembers] = useState<UserProfile[]>([]);

  const [issueModalOpen, setIssueModalOpen] =
useState(false);
  const [selectedUserId, setSelectedUserId] = useState("");
  const [selectedBookId, setSelectedBookId] = useState("");
  const [remarksInput, setRemarksInput] = useState("");

  const loadData = () => {
    setBorrows(DataStore.getBorrows());
    setBooks(DataStore.getBooks());
    setMembers(DataStore.getProfiles());
  };

  useEffect(() => {
    loadData();
    const handleDataChange = () => loadData();
    window.addEventListener("lms_data_change",
handleDataChange);
    return () =>
      window.removeEventListener("lms_data_change",
handleDataChange);
  }, []);

  // Filter for student/faculty view vs librarian view
  const userBorrows = isStudent || isFaculty
    ? borrows.filter((b) => b.user_id === currentUser?.id)

```

```

      : borrows;

      const activeBorrowsCount = userBorrows.filter((b) =>
b.status === "ACTIVE" || b.status === "OVERDUE").length;
      const availableSlots = Math.max(0,
(currentUser?.max_books_allowed || 0) - activeBorrowsCount);

      const handleIssueSubmit = (e: React.FormEvent) => {
e.preventDefault();
      if (!selectedUserId || !selectedBookId) {
        alert("Please select both a member and a book.");
        return;
      }

      const res = DataStore.issueBook(selectedUserId,
selectedBookId, remarksInput);
      alert(res.message);
      if (res.success) {
        setIssueModalOpen(false);
        setSelectedUserId("");
        setSelectedBookId("");
        setRemarksInput("");
        loadData();
      }
    };

    const handleReturn = (borrowId: string) => {
      if (confirm("Confirm return of this book?")) {
        const res = DataStore.returnBook(borrowId);
        alert(res.message);
        loadData();
      }
    };

    const handleRenew = (borrowId: string) => {
      const res = DataStore.renewBook(borrowId);
      alert(res.message);
      loadData();
    };

    return (
      <div className="space-y-5">
        /* Stat Cards Row */
        <div className="grid grid-cols-1 sm:grid-cols-3 gap-4">
          <div className="bg-white rounded-lg border border-slate-200 p-5 shadow-sm">
            <p className="text-3xl font-extrabold text-slate-900">{activeBorrowsCount} / {currentUser?.max_books_allowed || 0}</p>
            <p className="text-xs text-slate-500 font-semibold mt-1">Active Borrowed Books</p>
          </div>

          <div className="bg-white rounded-lg border border-slate-200 p-5 shadow-sm">
            <p className="text-3xl font-extrabold text-slate-900">{availableSlots}</p>
            <p className="text-xs text-slate-500 font-semibold mt-1">Available Slots</p>
          </div>

          <div className="bg-white rounded-lg border border-slate-200 p-5 shadow-sm">
            <p className="text-3xl font-extrabold text-slate-900">{userBorrows.length}</p>
            <p className="text-xs text-slate-500 font-semibold mt-1">Total History Records</p>
          </div>
        </div>

        /* Circulation / History Table Container */
        <div className="bg-white rounded-lg border border-slate-200 shadow-sm p-5 space-y-4">
          <div className="flex items-center justify-between pb-3 border-b border-slate-100">
            <h3 className="text-sm font-bold text-slate-900">Borrowing History</h3>

```

```

          <div className="flex items-center gap-2">
            {canIssueReturn && (
              <button
                onClick={() => setIssueModalOpen(true)}
                className="px-3.5 py-1.5 rounded bg-emerald-600 hover:bg-emerald-700 text-white text-xs font-semibold shadow-sm transition-all"
              >
                + Issue Book
              </button>
            )}
            <Link
              href="/books"
              className="px-3.5 py-1.5 rounded bg-blue-600 hover:bg-blue-700 text-white text-xs font-semibold shadow-sm transition-all"
            >
              Browse Catalog
            </Link>
          </div>

          <div className="overflow-x-auto">
            <table className="w-full text-left text-xs border-collapse">
              <thead>
                <tr className="bg-slate-50 text-slate-500 uppercase font-semibold border-b border-slate-200">
                  <th className="p-3">BOOK DETAILS</th>
                  {canIssueReturn && <th className="p-3">MEMBER</th>}
                  <th className="p-3">ISSUE DATE</th>
                  <th className="p-3">DUE DATE</th>
                  <th className="p-3">RETURN DATE</th>
                  <th className="p-3">RENEWALS</th>
                  <th className="p-3">STATUS</th>
                  <th className="p-3 text-right">ACTION</th>
                </tr>
              </thead>
              <tbody className="divide-y divide-slate-100 text-slate-700">
                {userBorrows.length === 0 ? (
                  <tr>
                    <td colspan={8} className="p-8 text-center text-slate-400">
                      No borrowing records found.
                    </td>
                  </tr>
                ) : (
                  userBorrows.map((b) => (
                    <tr key={b.id} className="hover:bg-slate-50/80 transition-colors">
                      <td className="p-3">
                        <div className="font-bold text-slate-900 text-xs">{b.book?.title}</div>
                        <div className="text-[11px] text-slate-500">
                          Author: {b.book?.author} | ISBN: {b.book?.isbn}
                        </div>
                      </td>
                      <td className="p-3">
                        {canIssueReturn && (
                          <td className="p-3">
                            <div className="font-bold text-slate-900">{b.user?.full_name}</div>
                            <div className="text-[11px] text-slate-500">{b.user?.role}</div>
                          </td>
                        )}
                      </td>
                      <td className="p-3 text-slate-600 font-mono">{b.issue_date}</td>
                      <td className="p-3 text-slate-600 font-mono">{b.due_date}</td>
                      <td className="p-3 text-slate-600 font-mono">{b.return_date || "-"}</td>
                      <td className="p-3 text-slate-600">{b.renewal_count} / 2</td>
                      <td className="p-3">
                        {b.status === "RETURNED" ? (

```

```

        className="w-full px-3 py-1.5 rounded
border border-slate-300 text-slate-900 focus:border-blue-600
bg-white"
        >
            <option value="">-- Select Member --
        </option>
        {members.map((m) => (
            <option key={m.id} value={m.id}>
                {m.full_name} ({m.role} - {m.email})
            </option>
        ))}
        </select>
    </div>

    <div>
        <label className="block font-semibold text-
        slate-700 mb-1">Select Book *</label>
        <select
            required
            value={selectedBookId}
            onChange={(e) =>
                setSelectedBookId(e.target.value)}
            className="w-full px-3 py-1.5 rounded
border border-slate-300 text-slate-900 focus:border-blue-600
bg-white"
            >
            <option value="">-- Select Available Book
            --</option>
            {books.filter((b) => b.available_copies >
            0).map((b) => (
                <option key={b.id} value={b.id}>
                    {b.title} ({b.available_copies}
                    available)
                </option>
            ))}
            </select>
        </div>

        <div>
            <label className="block font-semibold text-
            slate-700 mb-1">Remarks (Optional)</label>
            <input
                type="text"
                value={remarksInput}
                onChange={(e) =>
                    setRemarksInput(e.target.value)}
                placeholder="Standard checkout"
                className="w-full px-3 py-1.5 rounded
border border-slate-300 text-slate-900 focus:border-blue-
600"
            />
        </div>

        <div className="flex justify-end gap-2 pt-2
border-t">
            <button
                type="button"
                onClick={() => setIssueModalOpen(false)}
                className="px-3 py-1.5 rounded border
border-slate-300 text-slate-700 hover:bg-slate-50 font-
semibold"
            >
                Cancel
            </button>
            <button
                type="submit"
                className="px-4 py-1.5 rounded bg-blue-600
hover:bg-blue-700 text-white font-semibold shadow-sm"
            >
                Issue Loan
            </button>
        </div>
    </form>
</div>
</div>
    )}
</div>
);
}

```

Fine & Overdues (page.tsx)

```

"use client";

import React, { useState, useEffect } from "react";
import { useAuth } from "@context/AuthContext";
import { DataStore } from "@lib/data-store";
import { FineRecord } from "@lib/types";
import {
  DollarSign,
  AlertTriangle,
  CheckCircle2,
  Receipt,
  Search,
  CreditCard,
} from "lucide-react";

export default function FineOverduesPage() {
  const { currentUser, isLibrarian, isAdmin } = useAuth();
  const [fines, setFines] = useState<FineRecord[]>([]);
  const [searchQuery, setSearchQuery] = useState("");
  const [statusFilter, setStatusFilter] =
    useState<string>("ALL");
  const [selectedFine, setSelectedFine] =
    useState<FineRecord | null>(null);
  const [paymentModalOpen, setPaymentModalOpen] =
    useState(false);
  const [paymentMethod, setPaymentMethod] = useState("Credit
  Card");

  const loadData = () => {
    setFines(DataStore.getFines());
  };

  useEffect(() => {
    loadData();
    const handleDataChange = () => loadData();
    window.addEventListener("lms_data_change",
  handleDataChange);
    return () =>
    window.removeEventListener("lms_data_change",
  handleDataChange);
  }, []);

  const handleSettleFine = (action: "PAID" | "WAIVED") => {
    if (!selectedFine) return;
    DataStore.settleFine(
      selectedFine.id,
      action,
      action === "PAID" ? paymentMethod : undefined,
      action === "WAIVED" ? (currentUser?.full_name ||
  "Librarian") : undefined
    );
    setPaymentModalOpen(false);
    setSelectedFine(null);
    loadData();
  };

  const filteredFines = fines.filter((f) => {
    const matchesSearch =

f.user_name.toLowerCase().includes(searchQuery.toLowerCase()
) ||

f.user_email.toLowerCase().includes(searchQuery.toLowerCase()
) ||

f.book_title.toLowerCase().includes(searchQuery.toLowerCase()
));
    const matchesStatus = statusFilter === "ALL" ||
f.payment_status === statusFilter;
    return matchesSearch && matchesStatus;
  });

  const totalUnpaid = fines
    .filter((f) => f.payment_status === "UNPAID")
    .reduce((acc, f) => acc + f.amount, 0);

  const totalCollected = fines

```

```

    .filter((f) => f.payment_status === "PAID")
    .reduce((acc, f) => acc + f.amount, 0);

  const totalWaived = fines
    .filter((f) => f.payment_status === "WAIVED")
    .reduce((acc, f) => acc + f.amount, 0);

  return (
    <div className="space-y-5">
      {/* 4 Stat Cards */}
      <div className="grid grid-cols-1 sm:grid-cols-2
lg:grid-cols-4 gap-4">
        <div className="bg-white rounded-lg border border-
slate-200 p-5 shadow-sm">
          <p className="text-3xl font-extrabold text-rose-
600">${totalUnpaid.toFixed(2)}</p>
          <p className="text-xs text-slate-500 font-semibold
mt-1">Unpaid Balance</p>
        </div>

        <div className="bg-white rounded-lg border border-
slate-200 p-5 shadow-sm">
          <p className="text-3xl font-extrabold text-
emerald-600">${totalCollected.toFixed(2)}</p>
          <p className="text-xs text-slate-500 font-semibold
mt-1">Total Revenue Collected</p>
        </div>

        <div className="bg-white rounded-lg border border-
slate-200 p-5 shadow-sm">
          <p className="text-3xl font-extrabold text-blue-
600">${totalWaived.toFixed(2)}</p>
          <p className="text-xs text-slate-500 font-semibold
mt-1">Total Waived Fines</p>
        </div>

        <div className="bg-white rounded-lg border border-
slate-200 p-5 shadow-sm">
          <p className="text-3xl font-extrabold text-slate-
900">${fines.length}</p>
          <p className="text-xs text-slate-500 font-semibold
mt-1">Overdue Records</p>
        </div>
      </div>

      {/* Top Filter Bar */}
      <div className="bg-white p-4 rounded-lg border border-
slate-200 shadow-sm flex flex-col md:flex-row gap-3 items-
center justify-between">
        <div className="flex flex-1 flex-col sm:flex-row
gap-2.5 w-full">
          <input
            type="text"
            placeholder="Search member, email, book
title..."
            value={searchQuery}
            onChange={(e) => setSearchQuery(e.target.value)}
            className="flex-1 px-3 py-2 rounded border
border-slate-300 text-slate-900 text-xs focus:outline-none
focus:border-blue-600 bg-white"
          />

          <select
            value={statusFilter}
            onChange={(e) =>
              setStatusFilter(e.target.value)}
            className="px-3 py-2 rounded border border-
slate-300 text-slate-700 text-xs focus:outline-none
focus:border-blue-600 bg-white"
          >
            <option value="ALL">All Statuses</option>
            <option value="UNPAID">Unpaid</option>
            <option value="PAID">Paid</option>
            <option value="WAIVED">Waived</option>
          </select>
        </div>
      </div>

      {/* Fines Table */}

```

```

<div className="bg-white rounded-lg border border-
slate-200 shadow-sm overflow-hidden">
  <div className="overflow-x-auto">
    <table className="w-full text-left border-collapse
text-xs">
      <thead>
        <tr className="bg-slate-50 text-slate-500
uppercase font-semibold border-b border-slate-200">
          <th className="p-3">MEMBER DETAILS</th>
          <th className="p-3">BOOK TITLE & OVERDUE
INFO</th>
          <th className="p-3">FINE AMOUNT</th>
          <th className="p-3">STATUS</th>
          <th className="p-3 text-right">ACTIONS</th>
        </tr>
      </thead>
      <tbody className="divide-y divide-slate-100
text-slate-700">
        {filteredFines.length === 0 ? (
          <tr>
            <td colspan={5} className="p-8 text-center
text-slate-400">
              No fine records found.
            </td>
          </tr>
        ) : (
          filteredFines.map((fine) => (
            <tr key={fine.id} className="hover:bg-
slate-50/80 transition-colors">
              <td className="p-3">
                <div className="font-bold text-slate-
900">{fine.user_name}</div>
                <div className="text-[11px] text-
slate-500">{fine.user_email}</div>
              </td>
              <td className="p-3">
                <div className="font-bold text-slate-
900">{fine.book_title}</div>
                <div className="text-[11px] text-rose-
600 font-semibold">
                  {fine.days_overdue} Days Overdue
                  ({fine.reason})
                </div>
              </td>
              <td className="p-3 font-extrabold text-
slate-900 text-sm">
                ₹{fine.amount.toFixed(2)}
              </td>
              <td className="p-3">
                <span
                  className={`px-2.5 py-0.5 rounded
text-xs font-bold uppercase ${
                    fine.payment_status === "PAID"
                      ? "bg-emerald-50 text-emerald-
700 border border-emerald-200"
                      : fine.payment_status ===
"WAIVED"
                      ? "bg-blue-50 text-blue-700
border border-blue-200"
                      : "bg-rose-50 text-rose-700
border border-rose-200 font-bold"
                    }`}
                >
                  {fine.payment_status}
                </span>
              </td>
              <td className="p-3 text-right">
                {fine.payment_status === "UNPAID" &&
(isLibrarian || isAdmin) ? (
                  <button
                    onClick={() => {
                      setSelectedFine(fine);
                      setPaymentModalOpen(true);
                    }}
                    className="px-3 py-1 rounded bg-
blue-600 hover:bg-blue-700 text-white font-semibold text-xs
shadow-sm transition-all"
                  >
                    Settle Fine

```

```

                </button>
              ) : (
                <span className="text-[11px] text-
slate-400 font-medium">Settled</span>
              )
            </td>
          </tr>
        )
      </tbody>
    </table>
  </div>
</div>

{/* Settle Modal */}
{paymentModalOpen && selectedFine && (
  <div className="fixed inset-0 z-50 flex items-center
justify-center p-4 bg-black/50 backdrop-blur-xs">
    <div className="bg-white w-full max-w-sm p-6
rounded-lg border border-slate-200 shadow-xl space-y-4">
      <h3 className="text-base font-bold text-slate-
900 border-b pb-2">
        Settle Fine Payment
      </h3>

      <div className="p-3 bg-slate-50 rounded border
border-slate-200 text-xs space-y-1">
        <p><strong className="text-slate-
700">Member:</strong> {selectedFine.user_name}</p>
        <p><strong className="text-slate-
700">Book:</strong> {selectedFine.book_title}</p>
        <p><strong className="text-slate-700">Fine
Amount:</strong> ₹{selectedFine.amount.toFixed(2)}</p>
      </div>

      <div className="space-y-1 text-xs">
        <label className="block font-semibold text-
slate-700">Payment Method</label>
        <select
          value={paymentMethod}
          onChange={(e) =>
            setPaymentMethod(e.target.value)}
          className="w-full px-3 py-1.5 rounded border
border-slate-300 text-slate-900 focus:border-blue-600 bg-
white"
        >
          <option value="Credit Card">Credit / Debit
Card</option>
          <option value="Cash">Cash Counter
Settlement</option>
          <option value="University
Account">University Account Deduction</option>
        </select>
      </div>

      <div className="flex items-center justify-
between pt-3 border-t">
        <button
          type="button"
          onClick={() => handleSettleFine("WAIVED")}
          className="px-3 py-1.5 rounded border
border-slate-300 text-slate-700 hover:bg-slate-50 font-
semibold text-xs"
        >
          Waive Fine
        </button>

        <div className="flex gap-2">
          <button
            type="button"
            onClick={() => setPaymentModalOpen(false)}
            className="px-3 py-1.5 rounded border
border-slate-300 text-slate-700 hover:bg-slate-50 font-
semibold text-xs"
          >
            Cancel
          </button>
          <button
            type="button"

```

```

link.setAttribute("href", encodedUri);
link.setAttribute("download",
`LMS_Circulation_Report_${new
Date().toISOString().split("T")[0]}.csv`);
document.body.appendChild(link);
link.click();
document.body.removeChild(link);
});

return (
<div className="space-y-5">
  {/* 4 Stat Cards */}
  <div className="grid grid-cols-1 sm:grid-cols-2
lg:grid-cols-4 gap-4">
    <div className="bg-white rounded-lg border border-
slate-200 p-5 shadow-sm">
      <p className="text-3xl font-extrabold text-slate-
900">{books.length}</p>
      <p className="text-xs text-slate-500 font-semibold
mt-1">Total Catalog Titles</p>
    </div>

    <div className="bg-white rounded-lg border border-
slate-200 p-5 shadow-sm">
      <p className="text-3xl font-extrabold text-blue-
600">
        {Math.round(((totalCopies - availableCopies) /
(totalCopies || 1)) * 100)}%
      </p>
      <p className="text-xs text-slate-500 font-semibold
mt-1">Stock Utilization Rate</p>
    </div>

    <div className="bg-white rounded-lg border border-
slate-200 p-5 shadow-sm">
      <p className="text-3xl font-extrabold text-amber-
600">{activeBorrows}</p>
      <p className="text-xs text-slate-500 font-semibold
mt-1">Active Loan Count</p>
    </div>

    <div className="bg-white rounded-lg border border-
slate-200 p-5 shadow-sm">
      <p className="text-3xl font-extrabold text-slate-
900">{members.length}</p>
      <p className="text-xs text-slate-500 font-semibold
mt-1">Registered Members</p>
    </div>
  </div>

  {/* Action Buttons Header */}
  <div className="bg-white p-4 rounded-lg border border-
slate-200 shadow-sm flex items-center justify-between">
    <h2 className="text-sm font-bold text-slate-
900">Reports & Export Options</h2>

    <div className="flex items-center gap-2">
      <button
        onClick={exportCirculationCSV}
        className="px-3.5 py-1.5 rounded bg-blue-600
hover:bg-blue-700 text-white text-xs font-semibold flex
items-center gap-1.5 shadow-sm transition-all"
      >
        <Download className="w-3.5 h-3.5" />
        <span>Export CSV Report</span>
      </button>
      <button
        onClick={() => window.print()}
        className="px-3.5 py-1.5 rounded border border-
slate-300 text-slate-700 hover:bg-slate-50 text-xs font-
semibold transition-all"
      >
        Print
      </button>
    </div>
  </div>

  {/* Visual Charts & Role Breakdown */}

```

```

    <div className="grid grid-cols-1 lg:grid-cols-2 gap-
5">
      {/* Category Breakdown */}
      <div className="bg-white p-5 rounded-lg border
border-slate-200 shadow-sm space-y-4">
        <h3 className="text-sm font-bold text-slate-
900">Catalog Volume by Category</h3>

        <div className="space-y-3">
          {categoriesList.map((cat) => (
            <div key={cat.name} className="space-y-1">
              <div className="flex justify-between text-xs
text-slate-700 font-medium">
                <span>{cat.name}</span>
                <span className="text-slate-
500">{cat.count} Copies ({cat.percentage}%)</span>
              </div>
              <div className="w-full h-2 rounded-full bg-
slate-100 overflow-hidden">
                <div
                  className="h-full bg-blue-600 rounded-
full"
                  style={{ width: `${Math.max(5,
cat.percentage)}%` }}
                </div>
              </div>
            </div>
          ))}
        </div>
      </div>

      {/* Member Role Breakdown */}
      <div className="bg-white p-5 rounded-lg border
border-slate-200 shadow-sm space-y-4">
        <h3 className="text-sm font-bold text-slate-
900">User Roles Breakdown</h3>

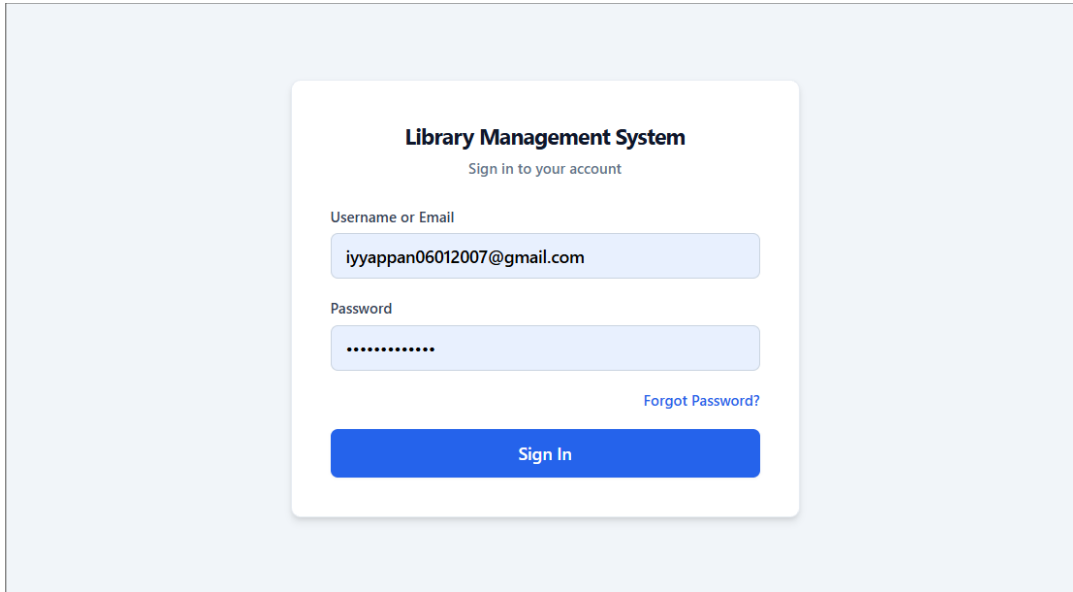
        <div className="grid grid-cols-2 gap-3">
          {[("STUDENT", "FACULTY", "LIBRARIAN", "ADMIN",
"COORDINATOR") as const].map((role) => {
            const count = members.filter((m) => m.role ===
role).length;
            return (
              <div key={role} className="p-3 rounded bg-
slate-50 border border-slate-200">
                <p className="text-xs text-slate-500 font-
semibold">{role}</p>
                <p className="text-xl font-bold text-
slate-900 mt-1">{count}</p>
              </div>
            );
          })}
        </div>
      </div>
    </div>
  );
}

```

Demonstration

This section demonstrates the working Library Management System. The deployed website for each module is shown below.

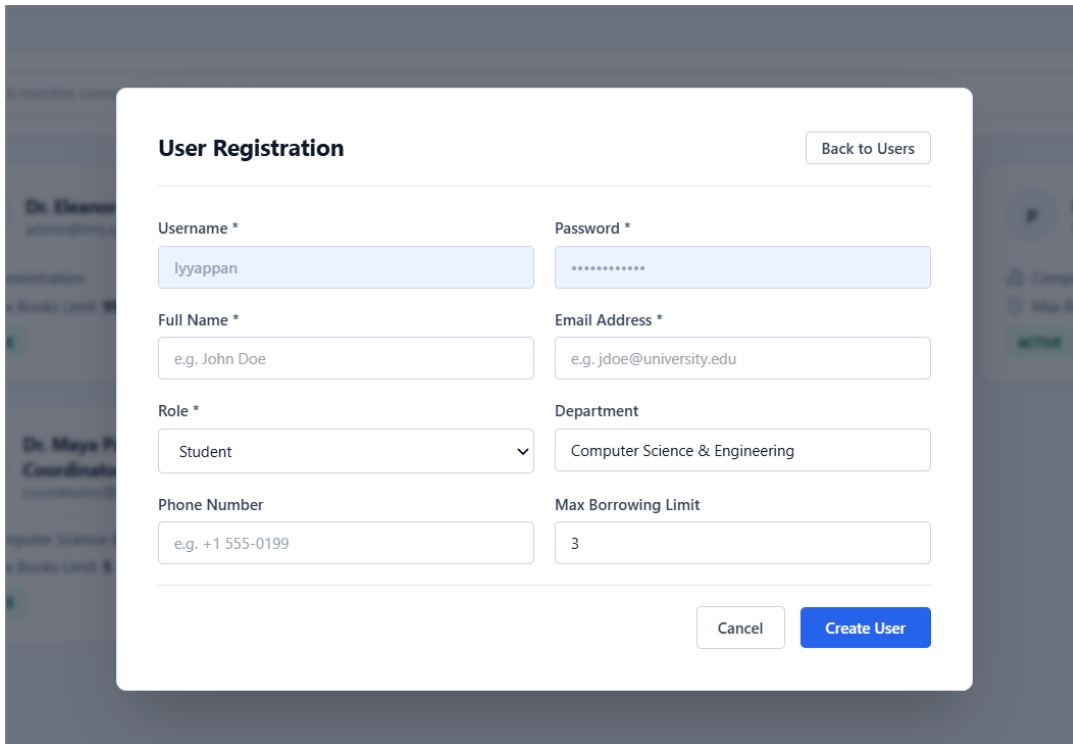
Login and Authentication



The screenshot shows a login form for the 'Library Management System'. The form is centered on a light blue background. It has a title 'Library Management System' and a subtitle 'Sign in to your account'. Below the subtitle, there are two input fields: 'Username or Email' with the value 'iyyappan06012007@gmail.com' and 'Password' with masked characters. A link 'Forgot Password?' is located below the password field. At the bottom of the form is a blue 'Sign In' button.

Figure 1: Login and Authentication

User and Profile Management



The screenshot shows a 'User Registration' form. The form is titled 'User Registration' and has a 'Back to Users' link in the top right corner. It contains several input fields: 'Username *' with the value 'iyyappan', 'Password *' with masked characters, 'Full Name *' with the value 'e.g. John Doe', 'Email Address *' with the value 'e.g. jdoe@university.edu', 'Role *' with a dropdown menu showing 'Student', 'Department' with the value 'Computer Science & Engineering', 'Phone Number' with the value 'e.g. +1 555-0199', and 'Max Borrowing Limit' with the value '3'. At the bottom right of the form are two buttons: 'Cancel' and 'Create User'.

Figure 2: User and Profile Management

Book Search and Catalog

The screenshot displays the 'Book Catalog' interface. On the left is a dark sidebar with navigation links: 'MAIN' (Dashboard), 'BOOKS' (Book Catalog), and 'MY LOANS & CIRCULATION' (My Borrowed Books). The main area has a header 'Book Catalog' with a subtitle 'Browse and manage the library book inventory'. Below this is a search bar with the placeholder 'Search by title, author, ISBN...' and filters for 'All Categories' and 'All Statuses'. A 'Search' button and a 'Reset' button are also present. The main content area shows 'Catalog Records (7)' with a 'Quick filter...' input. Below this is a table with 8 columns: ISBN, BOOK DETAILS, CATEGORY, PUBLISHER & YEAR, AVAILABILITY, SHELF, STATUS, and ACTIONS. The table lists 7 books with their respective details and status (AVAILABLE or CHECKED OUT).

ISBN	BOOK DETAILS	CATEGORY	PUBLISHER & YEAR	AVAILABILITY	SHELF	STATUS	ACTIONS
978-0132350884	Clean Code: A Handbook of Agile Software Craftsmanship by Robert C. Martin	Computer Science	Prentice Hall 1st Edition	4 / 6	CS-A-102	AVAILABLE	View
978-0262033848	Introduction to Algorithms (CLRS) by Thomas H. Cormen et al.	Computer Science	MIT Press 4th Edition	5 / 8	CS-B-204	AVAILABLE	View
978-1449373320	Designing Data-Intensive Applications by Martin Kleppmann	Computer Science	O'Reilly Media 1st Edition	0 / 7	CS-C-301	CHECKED OUT	View
978-0262035613	Deep Learning by Ian Goodfellow, Yoshua Bengio, Aaron Courville	Artificial Intelligence	MIT Press 1st Edition	3 / 5	AI-A-101	AVAILABLE	View
978-0073529325	Database System Concepts by Abraham Silberschatz et al.	Database Systems	McGraw-Hill 7th Edition	2 / 5	DB-A-101	AVAILABLE	View
978-0133594140	Computer Networks by Andrew S. Tanenbaum, David J. Wetherall	Networking	Pearson 5th Edition	4 / 4	NET-B-101	AVAILABLE	View
978-0134685991	Effective Java by Joshua Bloch	Computer Science	Addison-Wesley 3rd Edition	3 / 5	CS-A-105	AVAILABLE	View

Figure 3: Book Search and Catalog

Book Issue Management

The screenshot shows a modal dialog box titled 'Issue Book to Member'. It contains three main sections: 'Select Member *' with a dropdown menu showing '-- Select Member --'; 'Select Book *' with a dropdown menu showing '-- Select Available Book --'; and 'Remarks (Optional)' with a text input field containing 'Standard checkout'. At the bottom right are two buttons: 'Cancel' and 'Issue Loan'.

Figure 4: Book Issue Management

Book Return and Renewal

BOOK DETAILS	MEMBER	ISSUE DATE	DUE DATE	RETURN DATE	RENEWALS	STATUS	ACTION
Clean Code: A Handbook of Agile Software Craftsmanship Author: Robert C. Martin ISBN: 978-0132350884	Prof. Robert Thorne FACULTY	2026-09-18	2026-10-02	-	0 / 2	ACTIVE	<button>Renew</button> <button>Return</button>
Effective Java Author: Joshua Bloch ISBN: 978-0134685991		2026-09-01	2026-09-15	2026-09-28	0 / 2	RETURNED	Completed
Designing Data-Intensive Applications Author: Martin Kleppmann ISBN: 978-1449373320		2026-09-10	2026-10-10	-	1 / 2	ACTIVE	<button>Renew</button> <button>Return</button>
Introduction to Algorithms (CLRS) Author: Thomas H. Cormen et al. ISBN: 978-0262033848		2026-08-20	2026-09-03	2026-09-02	0 / 2	RETURNED	Completed

Figure 5: Book Return and Renewal

Borrowing History Management

LMSLibrary System Portal

MAIN

- Dashboard

BOOKS

- Book Catalog

MY LOANS & CIRCULATION

- My Borrowed Books

hyappan STUDENT Log Out

My Borrowed Books

Track your current loans and borrowing history

1 / 3

Active Borrowed Books

2

Available Slots

1

Total History Records

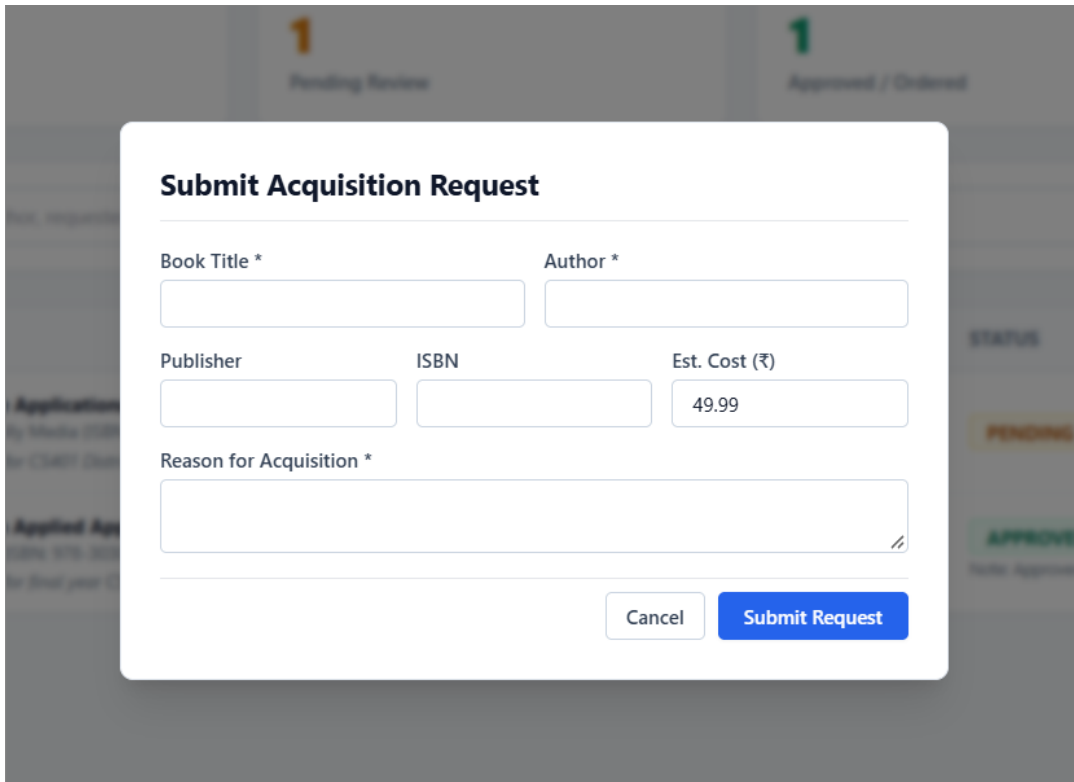
Borrowing History

Browse Catalog

BOOK DETAILS	ISSUE DATE	DUE DATE	RETURN DATE	RENEWALS	STATUS	ACTION
Deep Learning Author: Ian Goodfellow, Yoshua Bengio, Aaron Courville ISBN: 978-0262035613	2026-09-30	2026-10-14	-	0 / 2	ACTIVE	<button>Renew</button> <button>Return</button>

Figure 6: Borrowing History Management

Faculty Book Request



The image shows a 'Submit Acquisition Request' form overlaid on a blurred background of a library management system. The form is white with a dark header and contains several input fields and buttons.

Submit Acquisition Request

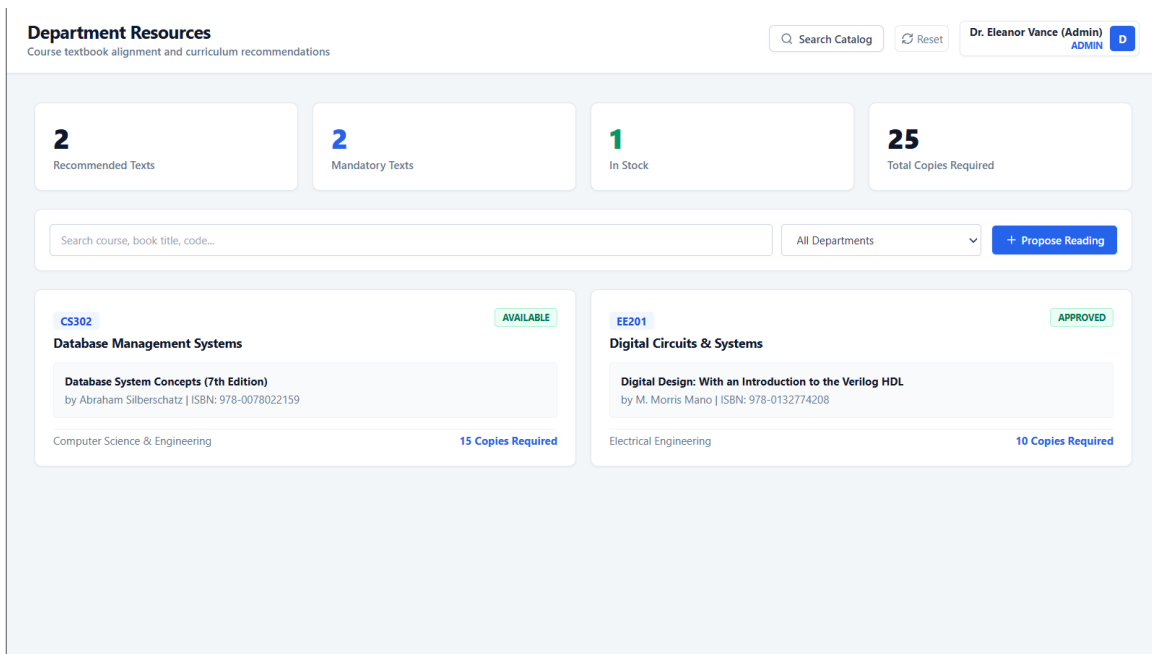
Book Title * Author *

Publisher ISBN Est. Cost (₹)

Reason for Acquisition *

Figure 7: Faculty Book Request

Department Resource Management



The image shows a 'Department Resources' dashboard. At the top, there's a header with 'Department Resources' and a subtitle 'Course textbook alignment and curriculum recommendations'. On the right, there are search and user controls. The main area features four summary cards: '2 Recommended Texts', '2 Mandatory Texts', '1 In Stock', and '25 Total Copies Required'. Below these is a search bar and a dropdown menu. The bottom section displays two book recommendations: 'Database Management Systems' (CS302) and 'Digital Circuits & Systems' (EE201), each with a status tag, title, author, and required copies.

Department Resources
Course textbook alignment and curriculum recommendations

Search Catalog Reset Dr. Eleanor Vance (Admin) ADMIN

2 Recommended Texts **2** Mandatory Texts **1** In Stock **25** Total Copies Required

Search course, book title, code... All Departments + Propose Reading

CS302 AVAILABLE
Database Management Systems
Database System Concepts (7th Edition)
by Abraham Silberschatz | ISBN: 978-0078022159
Computer Science & Engineering 15 Copies Required

EE201 APPROVED
Digital Circuits & Systems
Digital Design: With an Introduction to the Verilog HDL
by M. Morris Mano | ISBN: 978-0132774208
Electrical Engineering 10 Copies Required

Figure 8: Department Resource Management

Fine and Overdue Monitoring

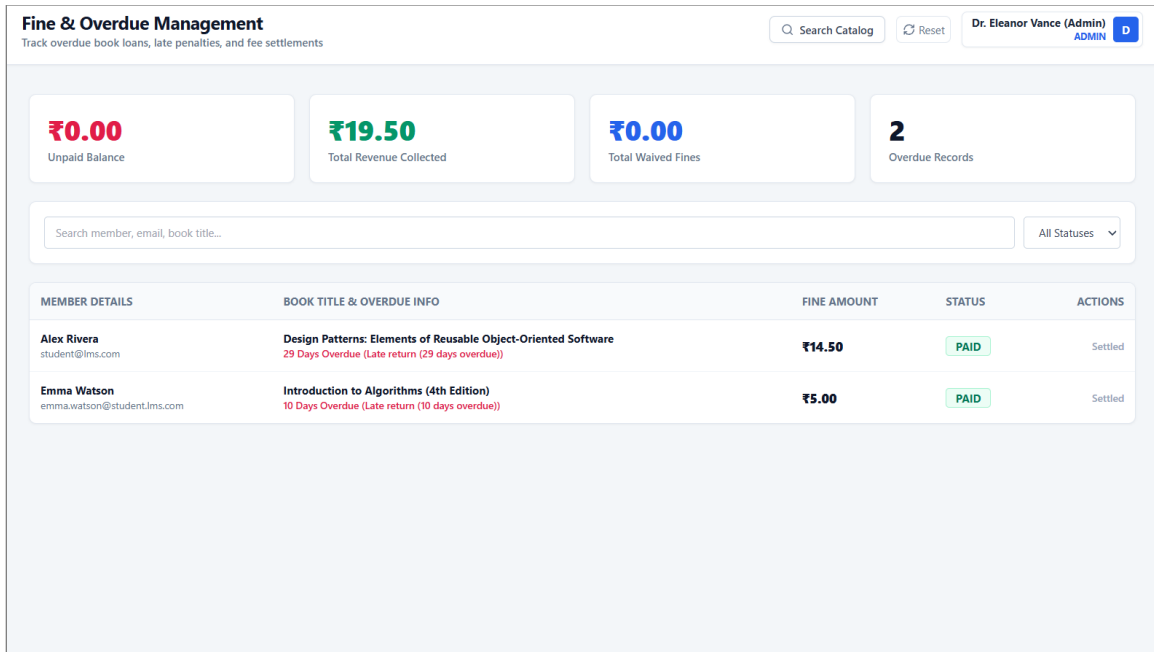


Figure 9: Fine and Overdue Monitoring

Reports and Analytics

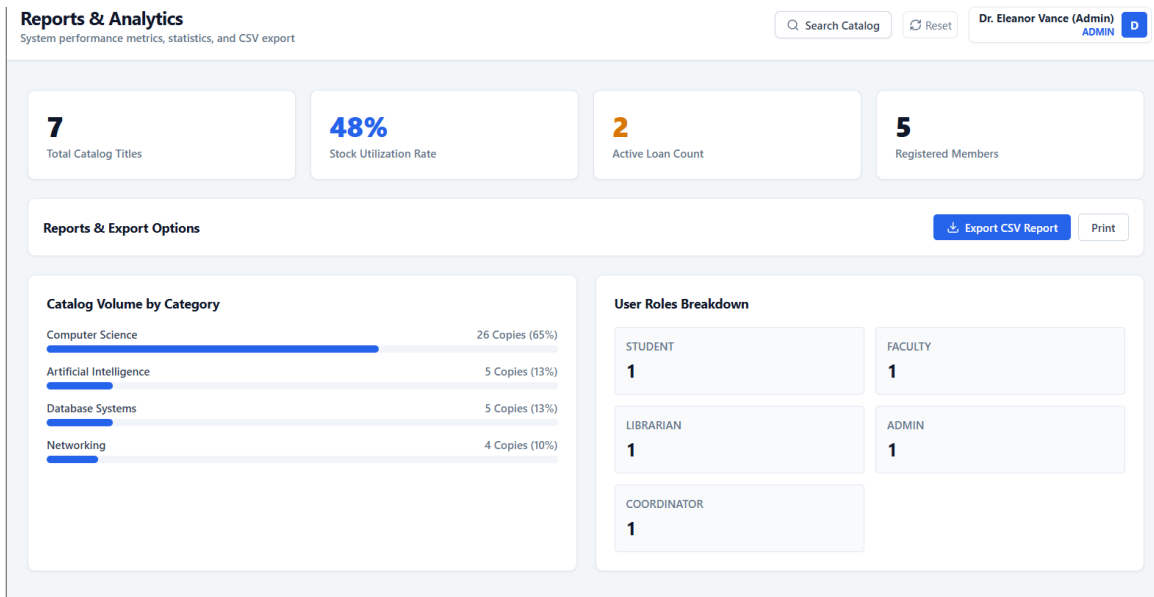


Figure 10: Reports and Analytics

E- Resources

Git Hub: <https://github.com/lyyappan06/library-management-system>

Appendix C: To Be Determined List

- **TBD-1:** *Final borrowing limits for Students and Faculty.*
- **TBD-2:** *Final renewal limits and renewal eligibility rules.*
- **TBD-3:** *Final fine calculation rate and overdue policies.*
- **TBD-4:** *Final report formats and templates for PDF and Excel exports.*
- **TBD-5:** *Final data backup and archival schedule for historical library records.*
- **TBD-6:** *Final configuration of notification requirements for borrowing, return, renewal, overdue, and request events.*